Certified Quantised Attention with Runtime Precision Escalation

Provable Error Bounds for Compressed KV Caches with Unconditional FP16 Fallback

Dean Calver

April 2026

Abstract

KV cache quantisation reduces the memory cost of long-context LLM inference, but introduces an approximation error whose magnitude is difficult to characterise in advance. Existing systems treat this as an acceptable tradeoff: compress aggressively, hope the model is robust.

We present a tiered KV cache that stores INT8 keys and INT4 values in GPU memory while retaining FP16 originals in system RAM. A two-term error decomposition provides independent, per-head, per-step bounds on key and value compression error. At runtime, the system monitors these bounds and adaptively selects precision per block: blocks with acceptably small error execute on compressed data; blocks that exceed the bound have their FP16 originals paged in. The worst case is not silent quality degradation—it is the Dense baseline's own `torch.scaled_dot_product_attention` code path with unmodified FP16 KV.

On LLaMA 3.1-8B across 4K–32K contexts, the certified system matches dense perplexity within noise on PG-19 and is statistically indistinguishable from dense on NIAH (100 paired trials, 8K). The system runs at $\sim$2.2$\times$ dense latency; effective VRAM savings require 128K+ context due to GQA working-set saturation at shorter lengths. No model retraining or dataset-specific calibration is required.

Contents

1 Introduction

Autoregressive LLM decoding at long context lengths is dominated by the cost of reading the key-value (KV) cache from GPU memory. Each decode step fetches cached keys and values for every attention head, a cost linear in sequence length. At 128K context on a model with 32 attention heads and head dimension 128, the KV cache alone occupies several gigabytes and consumes the majority of available memory bandwidth during decode.

KV cache quantisation [1, 2, 3, 4] addresses this by storing keys and values in reduced-precision formats—typically INT8, INT4, or lower. The savings are substantial: an INT8 cache halves memory usage, and INT4 quarters it. But quantisation introduces error, and the relationship between quantisation parameters and output quality is complex. The same quantisation scheme that works well on one prompt may fail on another, depending on the attention pattern, the key/value distributions, and the specific tokens being attended to.

The standard response is empirical validation: run a benchmark suite, check that perplexity doesn't degrade beyond a threshold, and deploy. This works in practice but provides no per-step guarantee. A system that achieves $\Delta\text{ppl} < 0.1$ on average may produce arbitrarily bad outputs on individual decode steps—and there is no mechanism to detect when this happens, let alone recover from it.

KV cache quantisation and compressed-domain attention are well-established techniques. What is missing is a *guarantee*: a way to bound, at each decode step, how far the quantised output deviates from an unquantised reference—and a way to recover the production dense output when the bound is too large.[1]

This paper provides that guarantee. The key ideas are:

1. Tiered storage with unconditional fallback. Quantised keys and values live in GPU memory; full-precision originals live in pinned system RAM. The FP16 data is not discarded after quantisation—it is retained as a safety net. Any block can be escalated to full precision at any time, at the cost of a host-to-device page-in from system RAM (overlappable with GPU compute). The worst case is the same `torch.scaled_dot_product_attention` code path and FP16 KV inputs used by the Dense baseline, not a degraded approximation.

2. Formal error bounds with runtime monitoring. A two-term error decomposition bounds the output perturbation from key compression and value compression independently. The key compression error depends on how quantisation shifts the softmax distribution; the value compression error depends on the per-token reconstruction quality. Both bounds are computable at runtime from quantities the system already tracks (quantisation scales, query norms, value ranges), enabling per-head, per-step precision decisions.

3. Adaptive per-block precision selection. Rather than applying a single quantisation policy uniformly, the system selects precision per block based on each block's importance to the current query. Blocks holding the majority of the attention mass use FP16 keys (paged in from system RAM); the remaining blocks use INT8. The selection adapts to the actual attention pattern at each decode step, providing FP16-quality results on the blocks that matter while retaining INT8 efficiency on the long tail.

[1] The paper distinguishes three reference outputs: O_{dense} (production Dense baseline via `torch.scaled_dot_product_attention`), O_{ref} (same Triton kernel with unquantised FP16 KV), and O_{quant} (Triton kernel with quantised KV). The formal bounds (Section 4) certify $\|O_{\text{quant}} - O_{\text{ref}}\|$; the arithmetic-path gap $\|O_{\text{ref}} - O_{\text{dense}}\|$ is characterised in Section 9.8. On fallback, the system returns O_{dense} directly.

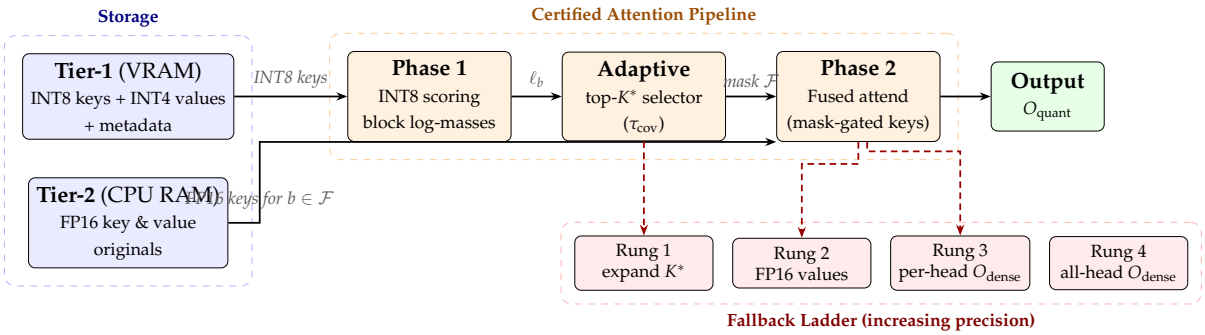

Figure 1: System overview. Tier-1 stores compressed KV in VRAM; Tier-2 retains FP16 originals in CPU RAM. Phase 1 computes INT8 block log-masses; the adaptive selector chooses top-K^* blocks for FP16 key promotion; Phase 2 fuses the mask-gated attend with INT4 value dequantisation. The fallback ladder (dashed red) escalates precision when monitors detect bound violations, with Rung 4 returning O_{dense} unconditionally.

1.1 Contributions

1. A **tiered KV cache architecture** that stores per-channel INT8 keys and per-group INT4 values in VRAM (approximately 56% of FP16 including metadata) while retaining FP16 originals in system RAM for runtime fallback.

2. A **two-term error decomposition** (key compression error + value compression error) with independent, per-head, per-step bounds computable at runtime.

3. An **adaptive precision selection mechanism** that uses the attention mass distribution (computed from quantised scores) to determine which blocks require FP16 keys, providing a guaranteed bound on key compression error.

4. A **ranking-consistency check** that detects when INT8 scoring noise distorts the attention distribution, triggering per-head fallback to dense attention. This addresses a non-obvious failure mode where non-uniform precision can cause attention dilution on retrieval-critical tokens.

5. A **four-rung fallback ladder** that escalates through increasingly conservative precision modes, with full FP16 recomputation as the unconditional terminal state.

What this paper does *not* claim. We do not claim end-to-end model-quality guarantees. The error bounds are *per-head, per-step*: they bound $\|O_{\text{quant}} - O_{\text{ref}}\|$ (quantised output vs. same kernel with FP16 KV; Section 4.1) for a single head at a single decode step. On fallback, the system returns O_{dense} directly. The aggregate effect on model quality is assessed empirically.

2 Background and Notation

2.1 Attention in Autoregressive Decoding

At decode step t, the model computes single-query attention over the cached key-value pairs:

$$O = \text{softmax}\left(\frac{q \cdot K^\top}{\sqrt{d}}\right) V \tag{1}$$

where $q \in \mathbb{R}^d$ is the current query, $K \in \mathbb{R}^{N \times d}$ the cached keys, $V \in \mathbb{R}^{N \times d}$ the cached values, and d the head dimension. This requires reading $2Nd$ elements from DRAM per head per step—the

dominant cost at long context.

2.2 Block Organisation

The KV cache is partitioned into contiguous blocks of B tokens (typically $B = 16$). Block b contains keys $\{k_t\}_{t \in b}$ and values $\{v_t\}_{t \in b}$. All precision decisions and error monitoring operate at block granularity. The block is the atomic unit of page-in from system RAM.

2.3 Quantisation Schemes

Per-channel INT8 keys. Each of the d key channels is quantised independently with its own scale s_c and zero point z_c:

$$k_{t,c}^{\text{int8}} = \text{clamp}\left(\text{round}\left(\frac{k_{t,c} - z_c}{s_c}\right), -128, 127\right), \qquad \hat{k}_{t,c} = k_{t,c}^{\text{int8}} \cdot s_c + z_c \tag{2}$$

Quantisation is applied to keys *after* RoPE (rotary position embeddings) has been applied, since LLaMA-family models store post-RoPE keys in the KV cache. The quantisation error bound Δ therefore accounts for any position-dependent magnitude variation introduced by RoPE.

Per-channel quantisation is critical for keys. Key channels have widely varying magnitudes (dynamic ranges spanning two orders of magnitude within a single head are common). Per-block quantisation (one scale for all channels) forces low-magnitude channels to share a scale set by the high-magnitude channels, crushing them to 1–2 INT8 levels. This produces measurable quality degradation on language modelling benchmarks. Per-channel quantisation, established by KVQuant [1] and adopted as standard practice, eliminates this by allowing each channel its own scale.

The per-channel scales and zeros are *per-block*: each block of B tokens has its own set of $2d$ FP32 values (scale + zero), computed once when the block is filled and never updated thereafter. Appended tokens accumulate in a trailing partial block that remains in FP16 until B tokens have arrived; at that point the block is quantised atomically with scales derived from those B tokens alone. This design eliminates the stale-scale problem: old blocks' INT8 codes always match their stored metadata, and the quantisation error Δ_b is block-local. The per-block metadata costs $2d \times 4/B = 64$ bytes per token at d=128, B=16, adding 50% to the INT8 key storage—but the keys are still half the size of FP16 ($16 \times 128 \times 2 = 4096$ bytes per block).

Per-group INT4 values. Each value vector of dimension d is divided into groups of g elements (default $g = 16$). Each group is independently quantised to INT4 with its own FP16 scale and zero point. Two INT4 values are packed into a single byte.

Per-group granularity is necessary for values because value vectors exhibit higher intra-vector dynamic range variation than keys. A single scale for the full d=128 vector would crush groups with small magnitudes. The group size g controls a quality–storage tradeoff: smaller groups provide finer-grained quantisation at the cost of more metadata. Empirically, g=16 achieves near-lossless quality ($\Delta\text{ppl} = +0.03$) with zero fallback escalation at the default tolerance. Smaller groups (g=4, 8) give the same quality but worse storage; larger groups (g=32) save marginally more memory but degrade catastrophically (Section 9).

The storage cost is $d/2$ bytes per token for the quantised values (INT4 packed), plus $2 \times (d/g) \times 2$ bytes per token for FP16 scales and zeros. For d=128 and g=16: 64 bytes (values) + 32 bytes (metadata) = 96 bytes per token, compared to 256 bytes for FP16. This is 37.5% of FP16.

Value error annotation. At quantisation time (cache write), the system computes the per-token ℓ_2 reconstruction error $\|\hat{V}_t - V_t\|_2$ and reduces it to a per-block maximum η_b. It also stores the per-block maximum value norm $\nu_b = \max_{t \in b} \|V_t\|_2$; the key error bound uses $V_{\max} = \max_b \nu_b$. Both annotations are one float per block, stored alongside the block in VRAM. Computing η_b

and ν_b at write time avoids the chicken-and-egg problem of needing FP16 originals to check quality at attend time.

FP16 originals. The unquantised FP16 keys and values are retained in pinned system RAM after quantisation. They are the ground truth for the attention computation and serve as the unconditional fallback when quantised precision is insufficient.

3 System Architecture

3.1 Tiered KV Cache

The KV cache is stored across two tiers, each serving a distinct role:

Tier 1 — VRAM (hot). Per-channel INT8 keys and per-group INT4 values, co-located with their quantisation metadata (per-channel scales/zeros for keys, per-group FP16 scales/zeros for values) and per-block value error annotations η_b. This is the data consumed during normal attention execution. Total storage per token is detailed in Section 9.5.

Tier 2 — CPU pinned RAM (cold). FP16 keys and values. The ground truth. Accessed when the adaptive precision selector promotes blocks to FP16 key precision, or when the value error annotation η_b indicates that a block's INT4 values exceed the configured tolerance. Page-in latency varies by interconnect: $\sim 3\,\mu$s per block over PCIe 5.0, lower over NVLink or CXL, effectively zero on unified-memory architectures. Optionally, a small VRAM scratch cache of recently paged-in FP16 blocks can amortise repeated page-in across consecutive decode steps; this does not affect correctness but reduces host-to-device transfer traffic in practice. Storage per token: 100% of FP16.

The key architectural decision is retaining Tier 2. Conventional systems discard FP16 originals after quantisation; by retaining them in system RAM, the system gains an unconditional escape hatch at the cost of ≈ 16 GB at 128K context (Section 11).

3.2 Fused Quantised Attention Kernel

The attention kernel executes directly on the quantised data in Tier 1, performing in-register dequantisation rather than staging through an FP16 buffer. This is the "compressed-domain execution" approach introduced by HACK [5] and VecInfer [6]: the INT8 keys are dequantised to FP16 in registers during the dot product, and the INT4 values are dequantised in registers during the weighted sum. No intermediate FP16 buffer is allocated.

For a single decode step:

1. For each block b, read INT8 keys from VRAM ($B \times d$ bytes).
2. Dequantise each key channel in registers: $\hat{k}_{t,c} = k^{\text{int8}}_{t,c} \cdot s_c + z_c$.
3. Compute attention logits: $s_{b,t} = q \cdot \hat{k}_{b,t} / \sqrt{d}$.
4. Compute block statistics: $m_b = \max_t s_{b,t}$ and per-block unnormalised mass.
5. Read INT4 values from VRAM, dequantise per-group in registers.
6. Accumulate into online softmax state: weights $\times$ values.

The scoring pass over all blocks produces per-block mass estimates used by the adaptive precision selector (Section 3.3). The selector determines which blocks require FP16 key precision; FP16 keys for those blocks are paged in from Tier 2 before the attend pass begins.

Fused scoring and attend. The system executes in two phases: a lightweight scoring pass to compute block masses and drive the adaptive selector, followed by a single fused attend pass

that processes all blocks with mask-gated key precision. Algorithm 2 gives the exact logic.

Algorithm 1: Fused Certified Attention with Mask-Gated Key Precision

Input: Query q; blocks $\{b_1, \ldots, b_N\}$ with INT8 keys + INT4 values in VRAM; top-K^* mask $\mathcal{F}$ from adaptive selector; FP16 keys for $\mathcal{F}$ in paged scratch buffer
Output: Attention output O

// — Phase 1: INT8 scoring (block log-masses for adaptive selector) —
for each block b **do**
Dequantise INT8 keys in-register: $\hat{k}_{t,c} = k_{t,c}^{\text{int8}} \cdot s_c + z_c$
Compute logits: $s_{b,t} = q \cdot \hat{k}_{b,t} / \sqrt{d}$
$m_b \leftarrow \max_t s_{b,t}$; $S_b \leftarrow \sum_{t \in b} \exp(s_{b,t} - m_b)$
$\ell_b \leftarrow m_b + \log S_b$ *// block log-mass (globally comparable)*
end for

// — Adaptive selection + async page-in —
$p_b \leftarrow \exp(\ell_b - \text{logsumexp}_j\, \ell_j)$ for all b *// normalised block mass*
$\mathcal{F} \leftarrow \text{AdaptiveTopK}(\{p_b\}, \tau_{\text{cov}}, K_{\min}, K_{\max})$
Page in FP16 keys for blocks $b \in \mathcal{F}$ from Tier 2 into scratch buffer

// — Phase 2: fused attend (single pass, mask-gated key precision) —
Initialise online softmax state: $m \leftarrow -\infty$ (FP64), $\ell \leftarrow 0$ (FP64), $o \leftarrow \mathbf{0}$ (FP32, d-dim)
for each block b **do**
// Load both key representations
$\hat{k}_b^{\text{int8}} \leftarrow$ dequantise INT8 keys for block b (always loaded)
if $b \in \mathcal{F}$: $k_b^{\text{fp16}} \leftarrow$ FP16 keys from scratch buffer
// Branchless precision selection

$$k_b \leftarrow \begin{cases} k_b^{\text{fp16}} & \text{if } b \in \mathcal{F} \\ \hat{k}_b^{\text{int8}} & \text{otherwise} \end{cases} \quad \text{(mask-gated } \texttt{where}\text{)}$$

// Score and accumulate (standard online softmax)
$s_{b,t} \leftarrow q \cdot k_{b,t} / \sqrt{d}$
$m_b^{\text{new}} \leftarrow \max_t s_{b,t}$
Update online softmax: m, ℓ, o with block scores and INT4 values $\hat{V}_b$
end for
$O \leftarrow o \,/\, \ell$

Figure 2: Fused certified attention with mask-gated key precision. Phase 1 is a lightweight scoring pass (INT8 keys only, no value reads) that computes per-block log-masses ℓ_b for the adaptive selector. Phase 2 is a single fused attend pass: each block's keys are selected from FP16 (promoted) or INT8 (rest) via mask-gated `where`. Values are INT4 for non-promoted blocks (Rung 2 promoted blocks use paged-in FP16 values). The online softmax scalars (m, ℓ) use FP64 precision; the output accumulator o uses FP32 (Section 9.8). Ranking-consistency checks occur after this kernel; value error E_{val} is computed post-attend from cached attention masses and pre-stored η_b.

3.3 Adaptive Precision Selection

After the scoring pass (Phase 1 in Algorithm 2), the system has per-block mass estimates from the INT8 scores. These drive the precision selection for the attend pass:

1. Sort all blocks by estimated attention mass (descending).
2. Compute cumulative mass fraction.
3. Find $K^* = \min\{k : C_k \geq \tau_{\text{cov}}\}$, where τ_{cov} is the coverage threshold (default 0.995).
4. Clamp: $K^* = \text{clamp}(K^*, K_{\min}, K_{\max})$.

5. Page in FP16 keys from Tier 2 for the top K^* blocks into a compact scratch buffer.
6. Execute the fused attend pass (Phase 2): each block uses either its FP16 keys (if promoted) or INT8 keys (otherwise), selected via a branchless mask.

The coverage threshold τ_{cov} controls the selector: blocks whose *INT8-estimated* masses sum to at least τ_{cov} of the total estimated mass are promoted to FP16 keys. The certified guarantee (Theorem 4.4) is on the *tail*: the true mass left to INT8 execution is at most $e^{2\Delta}(1 - \tau_{\text{cov}})$, regardless of whether the ranking is exactly correct. The implementation conservatively substitutes $e^{3\Delta}$ to accommodate optional INT8 query scoring (Section 4.4).

The coverage threshold $\tau_{\text{cov}} = 0.995$ means K^* adapts to the attention pattern: on concentrated heads (e.g., a single sink token holding most of the mass), K^* can be as low as $K_{\min}$; on diffuse heads (e.g., broad context gathering in early layers), K^* expands toward the cap. With $K_{\max}$=128 blocks, the maximum promoted fraction is 50% at 4K (256 blocks), 25% at 8K (512 blocks), and 12.5% at 16K (1024 blocks). Empirically, K^* frequently saturates at the cap for short contexts, where τ_{cov}=0.995 demands most blocks for coverage, and falls well below the cap at longer contexts where attention mass concentrates on a smaller fraction of blocks (Section 9). The cost is self-regulating: the system spends precision where the attention pattern demands it.

3.4 Fallback Ladder

When runtime monitoring detects that an error bound may be exceeded, the system escalates through increasingly conservative precision modes:

1. **Expand FP16 key coverage**: Increase K^* beyond the adaptive selection (e.g., double it), reducing the tail mass on INT8 keys.
2. **Promote values to FP16**: Before Phase 2, for blocks where $\hat{\rho}_b \cdot \eta_b$ (Phase-1 estimated mass $\times$ pre-stored reconstruction error) exceeds v_{tol}, page in FP16 values from Tier 2 (Corollary 4.2). The decision uses only pre-stored η_b and Phase-1 masses; no FP16 value reads are needed. After Phase 2, the exact $E_{\text{val}} = \sum_b \rho_b \eta_b$ is computed from actual attention masses and reported as telemetry; promoted blocks contribute η_b=0, so no recomputation is required.
3. **Ranking-consistency check**: After Phase 2, compare the top-r block ranking under FP16 block log-masses against the INT8 ranking from Phase 1, and verify that no tail block's upper-bounded FP16 log-mass could enter the top-r (Eq. 15). If either check fails (Section 6.1), recompute attention for the affected head using all-FP16 keys and values from Tier 2. This per-head fallback guarantees dense-equivalent output on the steps where INT8 scoring noise would distort the attention distribution.
4. **Full FP16 recomputation**: Page in all FP16 keys and values from Tier 2; execute standard dense FP16 attention for all heads via `torch.scaled_dot_product_attention` (the same code path used by the Dense baseline).

Both Rung 3 and Rung 4 bypass the fused Triton kernel entirely and call `torch.scaled_dot_product_attention`, returning O_{dense} (Section 4.1). The practical value of the system depends on Rungs 3 and 4 being rarely triggered, which is the subject of the empirical evaluation. The runtime cost of escalation is host-to-device page-in latency, overlappable with GPU compute on other blocks.

4 Formal Error Bounds

4.1 Two-Term Error Decomposition

Reference outputs. The paper distinguishes three attention outputs, each computed for a single head at a single decode step:

- O_{dense}: the output of `torch.scaled_dot_product_attention` (Flash Attention) with FP16 keys and values. This is the production Dense baseline.
- O_{ref}: the output of the *same Triton kernel* used by the certified system, but operating on unquantised FP16 keys and values. This isolates quantisation error from the arithmetic-path difference between the Triton kernel (CUDA-core FP32 dot products) and Flash Attention (tensor-core matmul).
- O_{quant}: the output of the Triton kernel operating on quantised (INT8 key / INT4 value) data. This is the system's fast-path output.

The formal bounds (this section) certify $\|O_{\text{quant}} - O_{\text{ref}}\|_2$, bounding the quantisation error with the arithmetic path held fixed. The arithmetic-path gap $\|O_{\text{ref}} - O_{\text{dense}}\|_2$ is characterised separately in Section 9.8. On fallback (Rungs 3 and 4), the system bypasses the Triton kernel entirely and returns O_{dense}, so the worst case is the Dense baseline—not merely O_{ref}.

The total quantisation error decomposes into two independent terms:

$$\|O_{\text{quant}} - O_{\text{ref}}\|_2 \leq \underbrace{E_{\text{key}}}_{\text{key compression}} + \underbrace{E_{\text{val}}}_{\text{value compression}} \tag{3}$$

Each term has an independent bound, an independent runtime monitor, and an independent escalation path through the fallback ladder. This independence is practically useful: key compression affects the *attention distribution* (which tokens are attended to), while value compression affects the *output quality* given a fixed distribution. The two error sources have different characteristics and different remedies.

4.2 Theorem 1: Value Compression Error

Theorem 4.1 (Value Compression Error)**.** *Let a_t be the softmax attention weights (computed with whatever key precision is in use). Let V_t be the original FP16 value and $\hat{V}_t$ the quantised value, with $\|V_t - \hat{V}_t\|_2 \leq \eta$ for all t in the attended set. Then:*

$$E_{\text{val}} = \|\sum_t a_t \hat{V}_t - \sum_t a_t V_t\|_2 \leq \eta \tag{4}$$

Proof. By the triangle inequality and the convexity of norms:

$$\|\sum_t a_t(\hat{V}_t - V_t)\|_2 \leq \sum_t a_t \|\hat{V}_t - V_t\|_2 \leq \eta \sum_t a_t = \eta$$

where the last equality uses $\sum_t a_t = 1$. □

Corollary 4.2 (Blockwise value compression error)**.** *Let $\rho_b = \sum_{t \in b} a_t$ be the attention mass on block b, and $\eta_b = \max_{t \in b} \|V_t - \hat{V}_t\|_2$ the per-block value error annotation. Then:*

$$E_{\text{val}} \leq \sum_b \rho_b \, \eta_b \tag{5}$$

This blockwise form is computed exactly at runtime: the system evaluates $E_{\text{val}}^{(h)} = \sum_b \rho_b^{(h)} \eta_b$ per head h at each decode step, using the per-block attention masses $\rho_b^{(h)}$ (already computed by

the adaptive selector) and the stored per-block error annotations η_b. Blocks where $\rho_b \eta_b > v_{\text{tol}}$ are promoted to FP16 values from Tier-2, and the achieved $E_{\text{val}}^{(h)}$ after promotion is reported as part of the per-step certificate. A stronger rule would promote blocks greedily by descending $\rho_b \eta_b$ until $E_{\text{val}}^{(h)} \leq \varepsilon_{\text{val}}$ (total value-error budget enforcement); we note this as a straightforward extension but retain the per-block threshold as the current escalation policy.

Properties of this bound.

The bound depends only on the worst-case per-token reconstruction error η, not on the number of tokens N. This is because the softmax weights form a convex combination: the weighted average of bounded errors is itself bounded, regardless of how many terms participate.

For INT4 per-group values with group size g=16, η is determined by the maximum within-group dynamic range across all value tokens. Empirically, η is small and stable across prompts (mean $\sim$0.05, max $\sim$0.18 for random Gaussian-distributed values).

The bound is tight: it is achieved when all per-token compression errors are perfectly aligned (pointing in the same direction). In practice, compression errors across tokens are approximately random in direction, so the actual error is much smaller than η.

Escalation path. When the quant-time error annotation η_b exceeds the configured threshold for a block (e.g., due to an outlier value token), the system pages in FP16 values from Tier 2 for the affected block.

4.3 Theorem 2: Key Compression Error

Key compression error is more subtle than value compression error because keys affect the attention distribution nonlinearly through the softmax.

Theorem 4.3 (Key Compression Error)**.** *Let a_t be the exact softmax weights (from FP16 keys) and a'_t the approximate softmax weights (from quantised keys). Let $V_{\max} = \max_t \|V_t\|_2$. Then:*

$$E_{\text{key}} = \|\sum_t a'_t V_t - \sum_t a_t V_t\|_2 \leq 2V_{\max} \cdot \text{TV}(a, a') \tag{6}$$

where $\text{TV}(a, a') = \frac{1}{2}\sum_t |a_t - a'_t|$ is the total variation distance between the true and approximate attention distributions.

Proof.

$$\begin{aligned}\|\sum_t (a'_t - a_t) V_t\|_2 &\leq \sum_t |a'_t - a_t| \cdot \|V_t\|_2 \\ &\leq V_{\max} \sum_t |a'_t - a_t| = 2V_{\max} \cdot \text{TV}(a, a')\end{aligned}$$

□

Bounding the total variation. The total variation between the exact and approximate softmax distributions depends on the per-token score errors $|\Delta_t| = |s_t - s'_t|$. We derive the bound for per-channel INT8 quantisation with channel scales $\{s_c\}$.

The per-channel quantisation error satisfies $|k_{t,c} - \hat{k}_{t,c}| \leq s_c/2$ for each channel c. The score error is:

$$|\Delta_t| = \frac{|q \cdot (k_t - \hat{k}_t)|}{\sqrt{d}} \leq \frac{1}{\sqrt{d}} \sum_{c=1}^{d} |q_c| \cdot \frac{s_c}{2}$$

Applying the Cauchy–Schwarz inequality to $\sum_c |q_c| s_c \leq \|q\|_2 \cdot \|s\|_2$ where $s = (s_1, \ldots, s_d)$:

$$|\Delta_t| \leq \frac{\|q\|_2 \cdot \|s\|_2}{2\sqrt{d}} \triangleq \Delta \tag{7}$$

Since scales are per-block (Section 2.3), Δ is strictly per-block: each block b has its own $\Delta_b = \|q\|_2 \cdot \|s^{(b)}\|_2/(2\sqrt{d})$ where $s^{(b)}$ is that block's scale vector. At runtime, a tighter per-block bound can be computed directly as $\Delta_b = \frac{1}{2\sqrt{d}} \sum_c |q_c| s_c^{(b)}$. For all downstream bounds (Theorems 4.4 and 4.3, Eq. 12), we use the conservative $\Delta = \max_{b \in \mathcal{T}} \Delta_b$ over the tail blocks $\mathcal{T}$, which is a single scalar per head per step and upper-bounds every block's individual Δ_b.

For per-channel quantisation, $\|s\|_2 = \sqrt{\sum_c s_c^2}$. When channel scales vary (as they do in practice), this is tighter than the per-block bound $\sqrt{d} \cdot s_{\max}$ because channels with small scales contribute less. For uniform scales ($s_c = \delta$ for all c), the bound reduces to $\Delta = \|q\|_2 \cdot \delta/2$, recovering the standard per-block result.

Bounding TV from score perturbation. The total variation between two softmax distributions whose logits differ by at most Δ can be bounded directly (Appendix A, Lemma A.2). For softmax distributions $a_t = \exp(s_t)/Z$ and $a'_t = \exp(s'_t)/Z'$:

$$\mathrm{TV}(a, a') \leq \tanh(\Delta) = \frac{e^{2\Delta} - 1}{e^{2\Delta} + 1} \tag{8}$$

This bound is tight, independent of sequence length N, and satisfies $\tanh(\Delta) \approx \Delta$ for small Δ.

In the adaptive precision system, the TV bound applies only to the *tail* blocks (those using INT8 keys). Let $\mathcal{F}$ denote the FP16 set and $\mathcal{T}$ the INT8 tail set. Since the FP16 blocks have zero key quantisation error, the key compression error decomposes as:

$$E_{\text{key}} \leq 2V_{\max} \cdot \mathrm{TV}_{\mathcal{T}}(a, a') \tag{9}$$

where $\mathrm{TV}_{\mathcal{T}}$ is the total variation restricted to the tail blocks. The tail blocks hold at most $(1 - \tau_{\text{cov}})$ of the *estimated* mass (by construction of the selector), and Theorem 4.4 ensures their true mass is at most $e^{2\Delta}(1 - \tau_{\text{cov}})$. The TV on the tail is therefore bounded by the product of the tail's mass fraction and the per-token score perturbation within the tail:

$$E_{\text{key}} \leq 2V_{\max} \cdot e^{2\Delta}(1 - \tau_{\text{cov}}) \cdot (e^{2\Delta} - 1) \tag{10}$$

At $\tau_{\text{cov}} = 0.995$ and $\Delta = 0.18$: $E_{\text{key}} \leq 2V_{\max} \cdot 1.43 \cdot 0.005 \cdot 0.43 \approx 0.006 V_{\max}$. The implementation substitutes $e^{3\Delta}$ throughout (giving $\approx 0.007 V_{\max}$) to accommodate optional INT8 query scoring (Section 4.4).

Escalation path. When the computed E_{key} exceeds a configured threshold, the system expands the FP16 set (increasing τ_{cov} and reducing the tail mass) or escalates through the fallback ladder.

4.4 INT8 Score Error for Mass Estimation

The adaptive precision selector uses INT8 scores to estimate per-block attention mass. We need to bound the error in these estimates to ensure the selector identifies the correct high-mass blocks.

Theorem 4.4 (INT8 Mass Estimation). *Let $S_b = \sum_{t \in b} \exp(s'_t - m'_b)$ and $m'_b = \max_{t \in b} s'_t$ be the block sum and block max computed from INT8 scores. Let $m'_{\text{global}} = \max_b m'_b$ be the INT8 global max. Then the true (FP16) unnormalised mass of block b satisfies:*

$$\hat{M}_b \leq S_b \cdot \exp(m'_b - m'_{\text{global}} + 2\Delta) \tag{11}$$

where Δ is defined in (7)*. The implementation uses $\exp(3\Delta)$ in place of the tight $\exp(2\Delta)$: the extra factor of e^{Δ} provides headroom for optional INT8×INT8 tensor-core scoring of the scaled query (Section 5.2), which would introduce a third error source. All theorems state the tight $\exp(2\Delta)$; the implementation's $\exp(3\Delta)$ substitution is noted where relevant.*

Proof. We bound $\hat{M}_b = \sum_{t \in b} \exp(s_t - m_{\text{global}})$ using INT8 quantities. Two error sources contribute:

Factor 1 (score perturbation): $|s_t - s'_t| \leq \Delta$ for each token, so $\exp(s_t) \leq e^{\Delta} \exp(s'_t)$.

Factor 2 (global-max shift): $m_{\text{global}} \geq m'_{\text{global}} - \Delta$, so $\exp(-m_{\text{global}}) \leq e^{\Delta} \exp(-m'_{\text{global}})$.

Combining:

$$\hat{M}_b = \sum_{t \in b} \exp(s_t - m_{\text{global}}) \leq e^{\Delta} \sum_{t \in b} \exp(s'_t - m_{\text{global}}) \leq e^{2\Delta} \sum_{t \in b} \exp(s'_t - m'_{\text{global}})$$

The identity $\sum_{t \in b} \exp(s'_t - m'_{\text{global}}) = S_b \cdot \exp(m'_b - m'_{\text{global}})$ is exact (algebraic re-expression, not an approximation), yielding:

$$\hat{M}_b \leq e^{2\Delta} \cdot S_b \cdot \exp(m'_b - m'_{\text{global}})$$

□

Numerical evaluation. For $d = 128$ (LLaMA 3.1-8B), typical per-channel quantisation gives $\Delta \approx 0.18$. The tight bound is $\exp(2\Delta) \approx 1.43$; the implementation uses $\exp(3\Delta) \approx 1.72$ to accommodate optional INT8 query scoring (Section 5.2).

What the upper bound certifies and what it does not. Theorem 4.4 provides a one-sided *upper* bound: the true mass of block b is at most $R_b = e^{2\Delta} \cdot S_b \cdot \exp(m'_b - m'_{\text{global}})$. This certifies a useful aggregate property: if the selector chooses blocks whose INT8-estimated masses sum to at least τ_{cov} of the total INT8-estimated mass, then the true mass of the *unchosen* blocks is at most $e^{2\Delta}(1 - \tau_{\text{cov}})$ of the total true mass.

However, the upper bound does *not* certify that the INT8 ranking is correct—block A with a higher INT8 estimate than block B is not guaranteed to have higher true mass. Both could have true masses anywhere between their INT8 estimates divided by $e^{2\Delta}$ and the INT8 estimates themselves. This means the selector may sometimes promote blocks that are not truly in the top-K^*. The consequence is benign: FP16 keys are used for a block that did not strictly need them, wasting a page-in but not introducing error. The guarantee that matters is on the *tail*: the total true mass left to INT8 is bounded, regardless of which specific blocks are in the FP16 set.

The fallback ladder (Section 3.4) provides the unconditional backstop: if the error from INT8 execution on the tail exceeds any configured threshold, the system expands the FP16 set or reverts to full FP16.

4.5 Total Error Bound

Combining the two terms:

$$\|O_{\text{quant}} - O_{\text{ref}}\|_2 \leq \underbrace{2V_{\max} \cdot e^{2\Delta}\, \hat{\alpha}_{\mathcal{T}}\, (e^{2\Delta} - 1)}_{E_{\text{key}}} + \underbrace{\textstyle\sum_b \rho_b\, \eta_b}_{E_{\text{val}}} \tag{12}$$

where $\hat{\alpha}_{\mathcal{T}} = \sum_{b \notin \mathcal{F}} p_b^{\text{int8}}$ is the INT8-estimated tail mass *after* all clamping and Rung 1 expansion, and $E_{\text{val}} = \sum_b \rho_b \eta_b$ is the achieved per-head value error (Corollary 4.2), computed exactly at runtime. The adaptive selector targets $\hat{\alpha}_{\mathcal{T}} \leq 1 - \tau_{\text{cov}}$, but $K_{\max}$ clamping may prevent this; the bound uses the achieved value, not the nominal threshold. By Theorem 4.4, the true tail mass satisfies $\alpha_{\mathcal{T}} \leq e^{2\Delta}\, \hat{\alpha}_{\mathcal{T}}$, so the bound is valid regardless of whether τ_{cov} was fully achieved. When the target is met ($\hat{\alpha}_{\mathcal{T}} = 1 - \tau_{\text{cov}}$), the bound reduces to $E_{\text{key}} \leq 2V_{\max} \cdot e^{2\Delta}(1 - \tau_{\text{cov}})(e^{2\Delta} - 1)$.

At $\tau_{\text{cov}} = 0.995$ and $\Delta = 0.18$, and assuming the target is met, the key term evaluates to $\approx 0.006\, V_{\max}$. The implementation substitutes $e^{3\Delta}$ throughout ($\approx 0.007\, V_{\max}$) to provide headroom for optional INT8 query scoring (Section 4.4). Both E_{key} and E_{val} are computed at runtime, independently monitorable, and independently escalatable through the fallback ladder.

4.6 Preconditions and Monitoring

The error bound (12) is valid under three preconditions. We distinguish between how each is *guaranteed* and how each is *monitored*:

P1: Quantisation metadata is current. Block scales and zeros must reflect the actual data in the block. *Guarantee:* deterministic by construction. Per-block, per-channel scales are computed once at block-fill time (Section 2.3) and are immutable thereafter; blocks are never partially overwritten. Appended tokens remain in FP16 in a trailing partial block until B tokens have arrived, at which point the block is quantised atomically.

P2: Quantisation error is bounded. The per-channel quantisation error for each token must satisfy $|k_{t,c} - \hat{k}_{t,c}| \leq s_c/2$. *Guarantee:* deterministic for round-to-nearest quantisation without saturation. Per-block min/max scales are chosen at block-fill time so that all values in the block are representable; since scales are computed from the block's own data range, saturation cannot occur by construction. *Monitoring:* a small exploration budget (Section 6) spot-checks dequantised values against FP16 originals as a defence-in-depth measure.

P3: No arithmetic overflow. The scoring and softmax computations must not overflow the accumulator range. *Guarantee:* deterministic for FP32 accumulators at $d \leq 256$ and typical score magnitudes; verified by standard overflow checks.

When any precondition is violated, the system escalates through the fallback ladder. Rung 4 (full FP16 recomputation via `torch.scaled_dot_product_attention`) is always available, making the system correct by construction. The overall guarantee is:

> **Contract.** For each head h and decode step t, the system either:
>
> 1. returns O_{quant} satisfying $\|O_{\text{quant}} - O_{\text{ref}}\|_2 \leq E_{\text{key}} + E_{\text{val}}$ (fast path, bounded relative to O_{ref}); or
> 2. returns O_{dense} via `torch.scaled_dot_product_attention` (Rung 3 per-head, Rung 4 all-heads; fallback path, exact Dense baseline).
>
> The system never returns an output with unknown or unbounded error. Both E_{key} and E_{val} are computed at runtime and available as per-step telemetry.

5 Compressed-Domain Execution

The fused kernel consumes INT8 keys and INT4 values directly, performing dequantisation in registers. This section analyses why this is preferable to the conventional approach of dequantising to an FP16 staging buffer.

5.1 Bandwidth Analysis

At decode time, attention is memory-bandwidth-bound. The cost is dominated by reading the KV cache from DRAM. For a single head at context length N with head dimension d:

Configuration	Bytes read	Relative
FP16 keys + FP16 values	$4Nd$	$1.0\times$
INT8 keys + FP16 values (dequant to buffer)	$3Nd$ + buffer write	$\sim 0.9\times$
INT8 keys + INT4 values (fused, in-register)	$1.5Nd$ + metadata	$\sim 0.4\times$

The fused in-register approach reads $\sim$40% of the data compared to FP16, with no intermediate buffer. The dequant-to-buffer approach saves on the read side but pays for the buffer write, partially negating the savings.

Note on the fused attend pass. The adaptive precision selector (Section 3.3) promotes K^* blocks

to FP16 key precision. The fused attend kernel reads both INT8 keys (all blocks, Nd bytes) and FP16 keys (promoted blocks only, $2K^*Bd$ bytes), selecting per block via a branchless mask. The total VRAM bandwidth is therefore $\sim 1.5Nd + 2K^*Bd$ bytes plus metadata, and the system RAM bandwidth is $2K^*Bd$ bytes for the page-in. When K^* is small relative to N/B (which it is for concentrated attention), the FP16 key reads add a modest fraction. The bandwidth table above characterises the INT8+INT4 baseline; Section 9.7 reports end-to-end decode throughput and per-step phase breakdown including the FP16 page-in overhead.

5.2 Per-Channel Key Scoring

Computing $q \cdot \hat{k}_t$ with per-channel quantisation expands as:

$$q \cdot \hat{k}_t = \sum_{c=1}^{d} q_c \cdot (k_{t,c}^{\text{int8}} \cdot s_c + z_c) = \underbrace{\sum_{c=1}^{d} (q_c \cdot s_c) \cdot k_{t,c}^{\text{int8}}}_{\text{modified dot product}} + \underbrace{\sum_{c=1}^{d} q_c \cdot z_c}_{\text{constant per block}} \tag{13}$$

The second term $\sum_c q_c z_c^{(b)}$ is constant across all tokens within block b (since zero points $z_c^{(b)}$ are per-channel within each block; see Section 2.3) and can be precomputed once per block per head per step. The first term is a standard dot product between the *scaled query* $\tilde{q}_c = q_c \cdot s_c$ and the INT8 key values. This can be computed using INT8 tensor cores (with the scaled query quantised to INT8) or FP16 (with the scaled query in FP16 and the INT8 keys widened per-element).

The choice between INT8 and FP16 scoring affects the scoring precision. FP16 scoring is exact given the dequantised keys, and the Δ bound in Equation (7) applies directly with two error sources (score perturbation and global-max shift), yielding $e^{2\Delta}$. INT8×INT8 tensor core scoring introduces a third error source: quantisation of the scaled query $\tilde{q}$. This increases the mass estimation bound from $e^{2\Delta}$ to $e^{3\Delta}$. The implementation uses $e^{3\Delta}$ throughout to accommodate this optional path without requiring re-derivation. All theorems in this paper prove the tight $e^{2\Delta}$ bound under FP16 query scoring; where the text uses $e^{3\Delta}$, it is the implementation's conservative substitution.

6 Instability Detection

Beyond the formal error bounds, the system includes runtime checks that detect when the quantised execution is producing anomalous results.

Score consistency check. During the second pass (FP16 keys for selected blocks), the system compares the FP16 attention scores against the INT8 scores from the first pass. If any token's score difference exceeds the declared bound, $|s_t^{\text{fp16}} - s_t^{\text{int8}}| > \Delta + \epsilon_{\text{guard}}$, the INT8 quantisation error has exceeded the precondition. This triggers Rung 4 fallback (full FP16 recomputation) for the affected head.

Exploration budget. A small fraction of blocks (1–5%) are randomly selected for FP16 key execution even when the adaptive selector would not choose them. The resulting scores are compared against the INT8 estimates, providing ongoing monitoring of the quantisation quality across the full cache, not just the high-mass blocks.

Value error annotations. Each block's maximum per-token reconstruction error η_b is computed at quantisation time and stored as a one-float annotation in VRAM. At attend time, the runtime computes the tight per-head value error $E_{\text{val}}^{(h)} = \sum_b \rho_b^{(h)} \eta_b$ (Corollary 4.2) and promotes blocks where $\rho_b \cdot \eta_b > v_{\text{tol}}$ to FP16 values from Tier 2. The achieved $E_{\text{val}}^{(h)}$ after promotion is reported as part of the per-step certificate. This avoids the need for runtime spot-checks against FP16

originals.

These checks are not necessary for the formal bounds to hold—the bounds are valid by construction. They provide defence in depth: catching cases where the quantisation parameters are stale, the cache has been corrupted, or the hardware is producing unexpected results.

6.1 Ranking-Consistency Check

The adaptive top-K selector uses INT8 scores (Phase 1) to determine which blocks receive FP16 keys in Phase 2. This creates a subtle vulnerability: the INT8 ranking may not match the true FP16 ranking. For two blocks with similar true scores, INT8 quantisation noise ($\sim$0.4% mean per-channel error) can swap their ordering. The formal mass bound (Theorem 4.4) guarantees that the *aggregate* tail mass is small, but does not guarantee that the *ranking* within the top-K^* is correct.

When the ranking is wrong, the consequences are task-dependent. For language modelling (where quality depends on the aggregate attention distribution), ranking errors on similar-mass blocks are benign—the output perturbation is within the certified bound. For retrieval tasks (where a single needle token must receive concentrated attention), ranking errors can matter: if a high-mass block is demoted to INT8 while a lower-mass block is promoted to FP16, the resulting mixed-precision softmax distribution may differ from the all-FP16 distribution in ways that affect the argmax over output logits.

The ranking-consistency check exploits data already available after Phase 2. For the K^* promoted blocks, the kernel has computed both INT8 scores (Phase 1) and FP16 scores (Phase 2). The check compares the top-r block ranking under both score sets:

$$\text{disagree}(h,t) = \mathbf{1}\Big[\text{argsort}(s^{\text{fp16}}_{1:K^*})_{1:r} \neq \text{argsort}(s^{\text{int8}}_{1:K^*})_{1:r}\Big] \tag{14}$$

where r is a configurable depth parameter (default r=1: only the top-1 block must agree).

When disagreement is detected for a head, the system escalates to Rung 3 of the fallback ladder: full FP16 recomputation for that head only. This returns O_{dense} by construction and is cheap when triggered rarely. The per-head granularity ensures that only the affected heads pay the cost; the remaining heads retain their fast-path results.

The detection cost is negligible: the FP16 scores are already computed during Phase 2 (the mask-gated kernel produces them as a byproduct of the `where` selection), and comparing two rankings of $\sim$100–200 elements requires microseconds. The kernel writes FP16 per-block scores to a side buffer at the cost of one additional global store per block per head.

Boundary verification. The ranking check as described compares rankings *within the promoted set* $\mathcal{F}$. A residual blind spot remains: a tail block $b \in \mathcal{T}$ (not promoted to FP16) might have a true FP16 block mass higher than some promoted blocks, but the check cannot observe this because the tail block's FP16 scores were never computed. To close this gap, the system performs an interval-bound verification on block log-masses.

Each block's log-mass from Phase 1 is $\ell_b^{\text{int8}} = m_b + \log \sum_{t \in b} \exp(s_{b,t}^{\text{int8}} - m_b)$. Since each token logit can shift by at most Δ under FP16 scoring, the FP16 block log-mass is bounded above by $\ell_b^{\text{ub}} = \ell_b^{\text{int8}} + \Delta$. Let $\ell_{(r)}^{\text{fp16}}$ denote the r-th highest FP16 block log-mass among promoted blocks. If any tail block satisfies:

$$\ell_b^{\text{int8}} + \Delta > \ell_{(r)}^{\text{fp16}} \tag{15}$$

then the ranking certificate cannot be issued for that head, and the system escalates to Rung 3 (per-head FP16 recomputation). This formulation ensures the boundary check matches the mass-based selection criterion used by the adaptive top-K selector and the mass-cover certificate

(Theorem 4.4). The check adds one comparison per tail block per head—negligible cost—and ensures the ranking certificate covers not just the promoted set's internal ordering but also the boundary between promoted and tail blocks. In practice, the adaptive selector's $\tau_{\text{cov}} = 0.995$ coverage threshold means tail blocks have very low mass, and their INT8 log-masses are typically far below the promoted set's r-th log-mass by a margin much larger than Δ. We observe zero boundary-check triggers across all benchmark runs (Table 7).

7 Related Work

7.1 KV Cache Compression

KVQuant [1] achieves sub-4-bit KV quantisation via per-channel key quantisation, establishing the per-channel technique we adopt for keys. **KIVI** [2] demonstrates tuning-free asymmetric 2-bit quantisation with the key observation that keys and values have different quantisation sensitivities—an observation central to our asymmetric INT8/INT4 design. **QJL** [3] uses Johnson–Lindenstrauss projections for 1-bit KV representations. **InnerQ** [4] introduces hardware-aware group-wise quantisation.

These systems treat compression as a storage optimisation: at decode time, compressed values are typically widened to FP16 before the attention kernel consumes them. Production serving frameworks (vLLM, TensorRT-LLM) now support FP8 KV caches with quantised-domain execution via FlashAttention-3, demonstrating that compressed-domain attention is practical at scale.

Recent work pushes compression further. **TurboQuant** [22] provides theoretically grounded online vector quantisation applicable to KV caches. **JanusQuant** [23] demonstrates accurate 2-bit KV cache quantisation for long-context inference. **PackInfer** [24] targets compute/IO-efficient attention kernels for packed quantised caches in batched serving. These optimise compression ratio and throughput; our contribution is orthogonal: runtime certification and exact fallback rather than more aggressive compression.

7.2 Compressed-Domain Execution

HACK [5] eliminates the dequantisation staging buffer by computing attention directly on quantised KV data, achieving significant JCT reduction. **VecInfer** [6] fuses dequantisation and attention into a single CUDA kernel. **FlashInfer** [7] provides infrastructure for page-organised KV caches with fused attention kernels.

Our work builds on this line but adds the formal error analysis and the tiered fallback architecture. Existing compressed-domain systems commit to a single quantisation level; our system adapts per block per step, with full-precision data available for escalation.

7.3 Adaptive Precision

Cocktail [17] selects precision per chunk. **TaDA** [18] adapts across layers. **Ada-KV** [19] dynamically allocates bits per token. **Don't Waste Your Bits** [20] formalises optimal bit allocation via attention entropy. These demonstrate that runtime precision adaptation is beneficial; our contribution is coupling adaptive precision with formal error bounds and an unconditional fallback guarantee.

7.4 Certified Bounds in Attention

Chen et al. [14] derive a mathematical theory of top-k sparse attention via total variation distance, including blockwise mass certificates for token selection—the closest prior work to our mass-estimation bounds. Our error decomposition differs in that it bounds output perturbation from *quantisation* (not token eviction), decomposing key compression and value compression

independently, and provides an unconditional FP16 fallback rather than a statistical guarantee.

7.5 KV Cache Memory Management

vAttention [15] uses CUDA virtual memory to manage KV cache allocation without the chunking overhead of PagedAttention, enabling contiguous KV storage with dynamic allocation. **PagedAttention** [16] (vLLM) introduced the paged KV cache paradigm. Our tiered storage is orthogonal: it concerns *precision* management across memory tiers, not *allocation* management within a single tier. The two approaches are complementary—a production system could use vAttention-style virtual memory for allocation within each tier.

8 Experimental Setup

Model. LLaMA 3.1-8B-Instruct with INT8 model weights (via `bitsandbytes` 8-bit quantisation), the standard production configuration for single-GPU deployment.

Hardware. NVIDIA RTX PRO 6000 (96 GB VRAM, Blackwell architecture). CPU: AMD EPYC 9355P (16 vCPU). System RAM: 282 GB. All experiments run single-GPU with pinned system RAM for Tier 2 storage.

Configurations. Two configurations are compared on every benchmark, using identical hardware, prompts, and random seeds:

Dense (baseline): FP16 KV cache with `torch.scaled_dot_product_attention` (dispatches to Flash Attention on supported hardware).

Certified (tiered): INT8 keys (per-channel) + INT4 values (g=16) in VRAM; FP16 originals in pinned system RAM. Adaptive top-K with τ_{cov}=0.995, $K_{\min}$=2, $K_{\max}$=128. Value tolerance v_{tol}=0.05. FP64 online-softmax accumulators. Ranking-consistency check with r=1.

Benchmarks. PG-19 (language modelling perplexity, 5 books, non-overlapping windows), NIAH (needle-in-a-haystack retrieval, 5 needles, 30 trials per context length by default, plus a powered 100-paired-trial follow-up at 8K), and RULER [21] (7 subtasks $\times$ 50 samples per context length). Context lengths: 4K, 8K, 16K, and 32K.

Metrics. Δ = Certified $-$ Dense for each metric. Per-run system statistics: K^* mean (adaptive top-K selection), value escalation rate (η-driven FP16 value promotion), ranking fallback rate (per-head ranking-consistency triggers).

9 Results

All results compare two configurations on the same hardware, prompts, and seeds: **Dense** (FP16 KV cache, the production baseline) and **Certified** (tiered quantised KV cache with adaptive precision selection and runtime fallback). Both use INT8 model weights.

9.1 Summary

Table 1 presents the headline quality results across all three benchmarks and context lengths from 4K to 32K. RULER and PG-19 show certified matching dense within noise at all tested contexts; at 32K (20 chunks), certified PG-19 perplexity is at parity with dense (Δppl$=-0.002$). On NIAH at 8K (100 paired trials), the certified system is statistically indistinguishable from dense ($\Delta=-2$pp, 95% CI $[-7, +3]$); the precision ablation in Section B decomposes the sources of variance.

Table 1: Quality parity: certified quantised attention vs. dense FP16 attention (LLaMA 3.1-8B). Δ = Certified $-$ Dense; positive means certified is better. All results use the hybrid mask-gated kernel with FP64 accumulators, $k_{\max}$=128, τ_{cov}=0.995, g=16, v_{tol}=0.05.

Benchmark	**Metric**	**4K**	**8K**	**16K**	**32K**	n
PG-19	Δ ppl	$+0.011$	$+0.003$	$+0.002$	-0.002	5 books / 20 chunks
NIAH	Δ accuracy	-6.7%	-2% $[-7, +3]$	$+3.3\%$	0.0%	30/100/30/30 trials
RULER	Δ accuracy	-0.001	$+0.004$	$+0.007$	$+0.002$	350 evals

9.2 PG-19 Perplexity

PG-19 evaluates language modelling perplexity on held-out book text, the standard quality gate for KV cache compression systems.

Table 2: PG-19 perplexity by context length. 4K–16K: 5 books, non-overlapping windows. 32K: 20 non-overlapping chunks with per-chunk CIs.

	4K	**8K**	**16K**	**32K**
Dense ppl	6.838	6.648	9.725	10.340
Certified ppl	6.849	6.651	9.726	10.338
Δ ppl	$+0.011$	$+0.003$	$+0.002$	-0.002

The certified system matches dense perplexity to three decimal places at 8K and 16K ($\Delta\text{ppl} \leq +0.003$). At 4K the delta is slightly larger ($+0.011$) but still negligible in absolute terms. At 32K (20 chunks), the delta is $\Delta\text{ppl}=-0.002$, within measurement noise—certified and dense are at parity.

Per-chunk confidence intervals. Higher-powered replications provide per-chunk variance estimates:

Table 3: PG-19 per-chunk replication. All Δppl values are at least two orders of magnitude smaller than the per-chunk variance ($\sim$5–7 ppl), confirming that the system preserves quality across diverse text.

Context	**Chunks**	**Δppl**	**Ratio**	**INT8-tail**
4K	20	$+0.009$	1.0009	59%
8K	20	-0.001	0.9999	55%
16K	20	$+0.002$	1.0002	73%
32K	20	-0.002	1.0000	86%

The INT8-tail fraction scales from 55% at 8K to 86% at 32K: the adaptive top-K serves an increasing majority of blocks at reduced precision as context grows, without measurable quality impact.

9.3 RULER

RULER [21] evaluates long-context capability across seven subtasks: four NIAH variants (single, multi-key, multi-value, multi-query), variable tracking, and two word-extraction tasks. Each configuration was evaluated with 50 samples per subtask.

Table 4: RULER accuracy by context length (50 samples × 7 subtasks = 350 evaluations per cell).

	4K	**8K**	**16K**	**32K**
Dense	0.955	0.930	0.898	0.886
Certified	0.955	0.933	0.905	0.888
Δ	−0.001	+0.004	+0.007	+0.002

RULER is the strongest result: the certified system matches or marginally exceeds dense at every tested context length from 4K to 32K, with $|\Delta| < 0.01$ throughout. All four NIAH subtasks within RULER achieve 100% accuracy for both dense and certified at 4K—the certified system introduces zero retrieval degradation on RULER's structured retrieval tasks.

Per-subtask confidence intervals. Using 20 paired samples per subtask per context length (n=140 total per context), the pooled paired 95% CI is $[-3.8\%, +2.4\%]$ at 4K (Δ=−0.5%) and $[-3.1\%, +1.7\%]$ at 8K (Δ=+0.6%)—both comfortably including zero. Per subtask, the retrieval variants (NIAH single/multi-key/multi-value/multi-query, CWE) show zero or near-zero deltas. Variable tracking (VT) shows Δ=−10pp at both 4K and 8K, but with CIs including zero ($[-29, +9]$ and $[-23, +3]$ respectively; 2–3 discordant trials). Few-word extraction (FWE) is noisy at both dense and certified baselines (45% and 20% respectively) with no systematic direction. No subtask shows a statistically significant certified degradation.

9.4 Needle-in-a-Haystack (NIAH)

NIAH evaluates single-fact retrieval at varying depth and context length, using 5-needle trials. An initial 30-trial run suggested a −6.7pp gap at 4K and 8K, but this corresponded to only 2 discordant trials—insufficient to distinguish from noise. We therefore ran a powered follow-up at 8K: 50 paired trials on the original 5 needles, plus 50 paired trials on 5 new (harder) needles, for 100 paired trials total.

Table 5: NIAH retrieval accuracy. 8K uses 100 paired trials (50 original + 50 harder needles) with bootstrap 95% CIs and McNemar test; 4K, 16K, and 32K use 30 trials (underpowered; CIs not reported). Absolute 8K accuracies are omitted because the two strata have very different baselines (original ∼93%, harder ∼6%); pooling obscures rather than clarifies. See text for per-stratum results.

	4K (n=30)	**8K** (n=100)	**16K** (n=30)	**32K** (n=30)
Dense	83.3%	—	70.0%	70.0%
Certified	76.7%	—	73.3%	70.0%
Δ	−6.7%	−2% $[-7, +3]$	+3.3%	0.0%
McNemar p	—	0.38	—	—

Main finding: the certified system is statistically indistinguishable from dense at 8K. The pooled 100-trial result at τ_{cov}=0.995 gives Δ=−2pp, 95% CI $[-7, +3]$, McNemar p=0.38. On the original 5 needles alone (50 paired trials), the point estimate is Δ=−6pp, 95% CI $[-16, +2]$—consistent with the earlier 30-trial run but with a CI that clearly includes zero. The 4K and 16K results (30 trials each) should be interpreted with the same caution: −6.7pp at n=30 is 2 trials and not statistically significant.

τ_{cov} sweep. We evaluated $\tau_{\text{cov}} \in \{0.990, 0.995, 0.999\}$ on the 100-trial 8K dataset. On the original 5 needles, all three operating points produce statistically indistinguishable results ($\Delta \in [-0.08, -0.06]$, all CIs including zero). On the 5 harder needles (where dense accuracy drops to 6%), the tighter threshold τ_{cov}=0.999 shows a positive trend: Δ=+8pp, bootstrap 95% CI $[+2, +16]$, but with only 4 discordant pairs the exact McNemar p=0.125 does not reach

conventional significance. This result is exploratory: the subset is small and should not be interpreted as a confirmed certified advantage, though the mechanism is plausible—tighter coverage forces more blocks to FP16 keys, reducing INT8 scoring noise at steps where the model's attention is most fragile.

The ranking-consistency check (Section 6.1) fires at ∼0.12% per head per step across all contexts, confirming that key ranking instability is rare under the hybrid mask-gated kernel. The 16K result (+3.3pp) should not be interpreted as a certified advantage; it is likely noise at n=30 where the base model's own retrieval accuracy is degraded (dense accuracy 70%).

A development-time NIAH precision ablation (Appendix B) decomposes the tiered–dense gap into three independent sources: INT4 value quantisation (∼6pp), INT8 key scoring on tail blocks (∼4pp), and kernel numerical path (∼4pp). The powered 100-trial evaluation at the operating point shows no statistically significant aggregate effect.

9.5 Storage Analysis

Table 6 breaks down the per-token **VRAM** storage cost for each component. This is the GPU memory footprint that determines how many tokens can be cached at a given VRAM budget. The Tier 2 system RAM cost (FP16 originals retained for fallback) is separate and equal to the dense FP16 cost.

Table 6: Per-token **VRAM** storage cost per KV head (d=128, block size B=16, value group size g=16). Tier 2 (CPU pinned RAM) stores FP16 originals at 512 bytes/token and is not included in this table.

Component	Bytes/token	vs. FP16	Notes
Dense FP16 (VRAM)			
Keys (FP16)	256	—	$d \times 2$
Values (FP16)	256	—	$d \times 2$
Total	**512**	**100%**	
Tier 1 — Certified (VRAM)			
Keys (INT8)	128		$d \times 1$
Key scales/zeros	64		$2d \times 4/B = 2 \times 128 \times 4/16$
Values (INT4)	64		$d/2$
Value scales/zeros	32		$2(d/g) \times 2 = 2 \times 8 \times 2$ (FP16)
Value η_b	< 1		1 float per block, amortised over B tokens
Tier 1 total	**288**	**56%**	

The Tier 1 VRAM cost is approximately 56% of dense FP16, a 44% VRAM reduction. The metadata overhead (scales and zeros) is significant—64 bytes per token for key metadata alone—but is amortised across the block and is the price of per-channel quantisation quality. The total memory footprint (VRAM + system RAM) is 288 + 512 = 800 bytes/token/head, or 156% of dense—the system trades system RAM for VRAM, which is the scarcer resource.

The Tier-1 savings are context-independent at 44%. For example, at 128K context (32 layers, 8 KV heads, d=128): dense FP16 KV requires ∼16 GB; Tier-1 requires ∼9 GB. However, the Tier-2 system RAM cost (FP16 originals for fallback) equals the dense size, and the runtime FP16 key scratch buffer further reduces the effective savings (see below).

Runtime FP16 key scratch buffer. At runtime, the system pages FP16 keys from Tier-2 into a VRAM scratch buffer to avoid repeated H2D transfers. The scratch buffer size determines the effective VRAM usage, and GQA union saturation (Section 9.7, Table 12) means it must be corpus-sized for reasonable throughput. The total runtime VRAM is therefore: Tier-1 (56% of dense) + union-sized FP16 key scratch (up to 50% of dense at the union fraction). At 64K context

with H_Q=32 query heads, the union covers 87% of blocks, so runtime VRAM is approximately $0.56 + 0.87 \times 0.50 = 1.0\times$ dense—no net saving. Meaningful VRAM savings emerge at 128K+ where the union fraction drops to 64% (Table 12).

To summarise the three memory tiers:

- **Tier-1 compressed KV** (VRAM): INT8 keys + INT4 values + metadata. 56% of dense, context-independent.
- **FP16 key scratch buffer** (VRAM): paged-in FP16 keys for the current working set. Size = union fraction $\times$ 50% of dense; context-dependent (Table 12).
- **Tier-2 originals** (system RAM): full FP16 KV retained for fallback. Equals dense cost. Required for the certification guarantee.

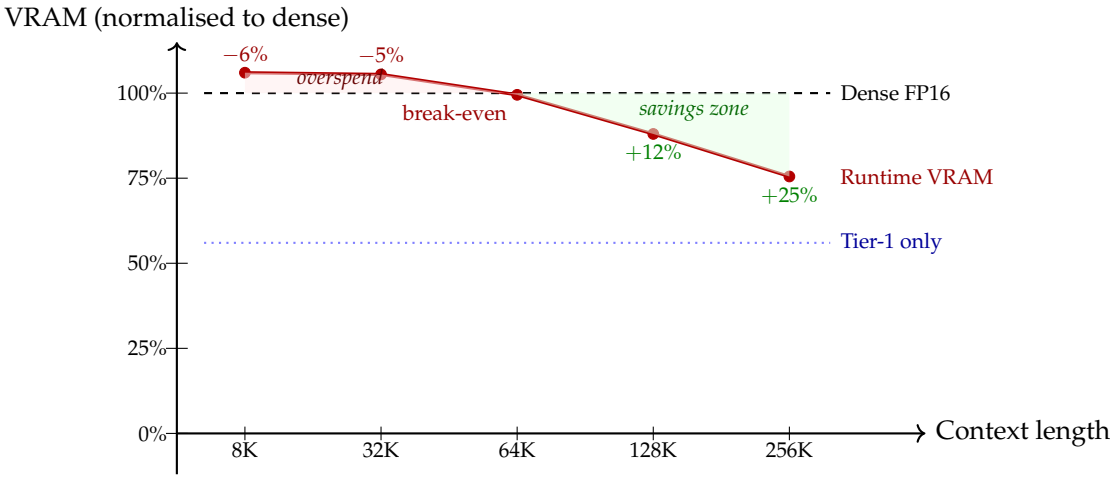

Figure 3: Runtime VRAM (Tier-1 + FP16 key scratch buffer) vs. dense FP16, normalised. Below ∼64K context, the GQA union saturates the corpus and the certified system uses *more* VRAM than dense. Meaningful savings (>10%) emerge at 128K+ where the working-set union shrinks (Table 12). The Tier-1 compressed storage (dotted blue) is always 56% of dense, but the runtime scratch buffer erodes this advantage at short contexts.

9.6 Quality Delta Summary

Table 7 summarises the quality delta across all twelve benchmark configurations. PG-19 and RULER show certified matching dense within noise ($|\Delta| < 0.012$) at all contexts from 4K to 32K. At 32K (20 chunks), certified PG-19 perplexity is at parity with dense (Δppl=-0.002) and NIAH ties at 70% accuracy (the model's observed retrieval performance at this context length). NIAH at 8K (100 paired trials) is statistically indistinguishable from dense; the 4K, 16K, and 32K cells are underpowered (n=30) and should be interpreted as directional only.

Table 7: Quality delta summary (Certified − Dense) across all configurations. Positive means certified outperforms dense. 8K NIAH shows 95% bootstrap CI from 100 paired trials; † marks underpowered cells (n=30).

Benchmark	4K	8K	16K	32K
PG-19 (Δppl)	+0.011	+0.003	+0.002	−0.002
NIAH (Δacc)	−6.7%†	−2% [−7, +3]	+3.3%†	0.0%†
RULER (Δacc)	−0.001	+0.004	+0.007	+0.002

System telemetry. The score-consistency canary (Section 6) registered zero violations across all 12 certified cells (1400 RULER evaluations, 190 NIAH trials, 20 PG-19 book-contexts), confirming that the Theorem 4.3 Δ bound was never empirically exceeded. The ranking-consistency fallback (Section 6.1) triggered at $\sim$0.12% per head per step on NIAH (flat across all contexts including 32K) and did not trigger on PG-19 or RULER. The boundary verification (Eq. 15) triggered zero times across all runs, confirming that the adaptive selector's coverage margin places tail blocks well below the promoted set's log-mass threshold. Rung 1 expansion (tail mass exceeds threshold after $K_{\max}$ clamping) shows strong benchmark dependence: it triggers on $\sim$100% of steps for NIAH and RULER but only 0.06% of steps for PG-19. The mechanism is not that coverage is hard on retrieval tasks—concentrated attention should make it easier—but that the conservative $e^{3\Delta}$ inflation of estimated tail mass (Section 4.4) makes the certified tail bound exceed the threshold even after selecting $K_{\max}$ blocks, triggering expansion to $2K_{\max}$. On PG-19, where attention is diffuse, $K_{\max}$=128 comfortably covers τ_{cov}=0.995 even under conservative inflation. This asymmetry does not affect quality: Rung 1 increases K^* to improve coverage, and the achieved tail mass $\hat{\alpha}_{\mathcal{T}}$ (which enters the bound) is always computed after expansion.

9.7 Runtime Performance

The quality results above demonstrate that the certified system matches dense accuracy. This subsection quantifies the runtime cost of certification, traces the optimisation from an initial 9.4$\times$ gap to 2.2$\times$ via kernel and cache improvements, and reports throughput across context lengths from 8K to 64K.

8K phase breakdown (pre-optimisation kernel). To identify optimisation targets, we instrumented 500 certified decode steps at 8K with the initial CUDA-core Triton kernel and sub-corpus cache (capacity = 64 blocks). This profiling motivated the kernel changes in Table 10.

Table 8: Per-step phase breakdown (500 certified decode steps, 8K NIAH, pre-optimisation CUDA-core kernel, cache capacity = 64). H2D page-in dominated at this operating point, motivating the kernel and cache optimisations in Table 10.

Phase	% of step
H2D page-in (FP16 keys from system RAM)	58.8%
Phase 2: fused attend (mask-gated key + INT4 value dequant + softmax)	15%
Non-attention (MLP, norms, projections)	18%
Adaptive selection (logsumexp, top-K, mask)	3%
Ranking-consistency check	2%
Phase 1: INT8 scoring (block log-masses)	2%
Value escalation check (η_b threshold)	0%

At 8K with sub-corpus cache, H2D page-in dominates at 58.8% of step time. All three benchmarks exhibit the same profile at sub-corpus cache sizes: $\sim$2–3% hit rate, $\sim$475 MB/step H2D transfer, regardless of prompt content or decode mode.[2]

- Rung 4 (full FP16 recompute) fire rate = 0.00% across all benchmarks, confirming the score-consistency bound holds with 5.5$\times$ headroom.
- Quality cross-check: $\Delta\text{ppl} = -0.005$ (PG-19), $\Delta\text{acc} = -0.067$ (NIAH, n=30), $\Delta\text{acc} = -0.02$ (RULER)—consistent with the main quality sweep, confirming the bounded cache does not degrade quality.

FP16 key cache capacity sweep (8K). Table 9 shows how throughput scales with cache capacity

[2]We verified this by running both teacher-forced and argmax decode on PG-19 and NIAH at cap = 64; all four combinations produce hit rates within 1 standard deviation of each other.

at 8K context.

Table 9: FP16 key cache capacity sweep (NIAH 8K, certified full system). Corpus size at 8K with block_size $= 16$ is 512 blocks. "Full mirror" retains all FP16 keys in VRAM.

Capacity	Note	tok/s	Hit rate	H2D MB/step
0	no cache	2.77	0.0%	486.6
64		3.06	2.6%	473.9
256		2.82	3.5%	469.6
384		3.03	5.7%	458.8
512	$= \lceil N/b \rceil$	6.51	99.6%	1.9
640		6.66	99.6%	1.9
768		6.57	99.6%	1.9
1024		6.51	99.6%	1.9
∞	full mirror	8.27	—	0.0

The sweep reveals a step function at exactly the corpus size. Below the corpus, the promoted set shifts across 32 layers at each step, thrashing the cache; at or above the corpus, the cache absorbs the entire FP16 key working set and hit rates jump from 5.7% to 99.6%. The remaining gap from corpus-capacity (6.51 tok/s) to full mirror (8.27 tok/s) is Python-level LRU overhead ($O(N)$ eviction), not H2D traffic.

Kernel optimisation. The 8K profiling above identified the Triton attention kernel (CUDA-core FP32 dot products, $\sim$10$\times$ slower than Flash Attention's tensor-core matmul) and the Python-level LRU cache as the dominant costs. Three targeted optimisations, applied at 64K full-mirror (no H2D), collapsed the gap from 9.4$\times$ to 2.1$\times$ (Table 10).

Table 10: Optimisation progression at 64K full-mirror (LLaMA 3.1-8B, RTX PRO 6000, 128 timed decode steps). Each row adds one change cumulatively.

Change	ms/step	tok/s	vs. dense
Original kernel (CUDA-core FP32)	478.0	2.09	9.4$\times$
+ Split-K kernel (FlashDecoding partition)	125.6	7.96	2.5$\times$
+ $O(1)$ OrderedDict LRU	123.6	8.09	2.5$\times$
+ Drop call-site `.contiguous()` copies	105.1	9.52	2.1$\times$
Dense (torch SDPA / Flash Attention)	50.9	19.65	1.0$\times$

The split-K kernel partitions the sequence across thread blocks (analogous to FlashDecoding [7]), maintains FP32 online-softmax state within each partition, and reduces across partitions in a second pass. This replaces the single-partition CUDA-core kernel with a parallelised implementation that better utilises GPU resources at long contexts. The `.contiguous()` removal eliminated 25 ms/step of `aten::copy_` that the profiler identified as the top self-CUDA consumer. None of these changes affect the error bounds or fallback logic.

Context-scaling throughput. Table 11 reports throughput across context lengths using the optimised kernel with full-mirror FP16 keys (all FP16 keys retained in VRAM, eliminating H2D traffic).

Table 11: Decode throughput vs. context length (LLaMA 3.1-8B, RTX PRO 6000, full mirror, 32 decode steps, Paper-1 hybrid with τ_{cov}=0.995). "INT8-tail" is the fraction of blocks served with INT8 keys (not promoted to FP16).

Context	Dense ms	Cert ms	Ratio	INT8-tail
8K	35.1	83.3	2.4×	58.6%
16K	37.3	82.3	2.2×	76.7%
32K	40.8	90.6	2.2×	87.0%
48K	41.3	90.3	2.2×	87.9%
64K	41.5	91.1	2.2×	87.9%

The certified overhead is approximately constant at ∼2.2× from 16K to 64K: certified step time grows slowly (82 → 91 ms), while dense step time grows at a similar rate (37 → 42 ms). The INT8-tail fraction climbs from 59% at 8K to 88% at 64K as the adaptive top-K^* set becomes a decreasing fraction of the corpus. At 8K the ratio is slightly worse (2.4×) because the INT8-tail is only 59%—more blocks require FP16 key treatment—and the corpus is small enough that all 32 layers' promoted sets overlap substantially in the cache.

Remaining overhead. The 2.2× residual at 64K decomposes as: (1) the split-K kernel is faster than the original CUDA-core kernel but still not tensor-core-native (∼1.5–1.8× estimated kernel gap); (2) Phase-1 INT8 scoring (∼10 ms/step, fusible with the attend kernel but not yet fused); (3) Python-level orchestration (adaptive selection, mask construction, small elementwise kernels totalling ∼40 ms). A tensor-core kernel with fused Phase-1 scoring would target ∼1.5× dense; eliminating the Python orchestration overhead would bring it closer to 1.3×.

GQA union saturation. The cache sweep's step function at corpus size is a structural consequence of GQA head diversity, not a cache-policy failure. Each of H_Q=32 query heads independently selects its own top-K^* blocks per layer. The FP16 key cache is shared across all KV groups within each layer, so the per-layer working set is the union across all 32 query heads (not the G=4 heads within a single KV group). With Rung 1 expansion, the effective K^* per query head is $\min(2K_{\max}, \lceil N/B \rceil)$. The per-layer FP16 working set is:

$$U = \lceil N/B \rceil \cdot \left(1 - \left(1 - \frac{\min(2K_{\max},\, \lceil N/B \rceil)}{\lceil N/B \rceil}\right)^{H_Q}\right) \tag{16}$$

Table 12 evaluates this for LLaMA 3.1-8B with $K_{\max}$=128, H_Q=32, B=16, and Rung 1 active (effective per-head selection $K^*_{\text{eff}} = \min(256, \lceil N/B \rceil)$). VRAM saving is Tier-1 (56% of dense) plus the union-sized FP16 key cache ($U \times$ 50% of dense), compared against dense FP16 KV.

Table 12: GQA union analysis (H_Q=32 query heads). Empirically confirmed at 64K: cache hit rate at sub-corpus capacities is <1%, consistent with the 87% union prediction.

Context	Blocks $\lceil N/B \rceil$	K^*_{eff}/N_B	Union	Dense (MB)	Saving
8K	512	50%	≈100%	1024	−6%
32K	2048	12.5%	99%	4096	−5%
64K	4096	6.2%	87%	8192	+1%
128K	8192	3.1%	64%	16384	+12%
256K	16384	1.6%	39%	32768	+25%

The corrected union analysis reveals that VRAM savings are negligible at 64K (break-even) and only become substantial (>10%) at 128K+ context. Below 64K, the union covers virtually

the entire corpus and the architecture uses *more* VRAM than dense. This is the central scaling challenge: the architecture's VRAM value proposition requires long enough contexts that $K^*_{\text{eff}}/\lceil N/B \rceil$ is small enough to overcome the H_Q=32 fan-out in Eq. 16.

Per-KV-group selection. As a targeted experiment (pre-optimisation kernel), we collapsed the 32 independent per-query-head selections into 8 per-KV-group selections (union within each group of G=4 query heads, then page in only the group union). At 64K with sub-corpus cache (cap = 1024): per-KV-group reduced H2D volume by 11% and improved throughput by 24%. This confirms the GQA fan-out hypothesis—fewer independent selectors produce a smaller union—but the improvement was modest because the kernel gap, not H2D, dominated. With the optimised kernel (Table 11), the full-mirror configuration eliminates H2D entirely, so per-KV-group selection is primarily relevant for sub-corpus deployments where VRAM is constrained.

Remaining optimisation targets. The 2.2× overhead is dominated by the attention kernel and Python orchestration, not the certification logic. Remaining targets, in priority order:

1. **Tensor-core kernel.** The split-K kernel still uses CUDA-core dot products. A tensor-core-native implementation (e.g., FlashInfer [7] with quantised-KV support) would close the remaining ∼1.5–1.8× kernel gap.
2. **Fused Phase-1 scoring.** The INT8 scoring pass (∼10 ms/step) reads the same keys as Phase-2; fusing both passes eliminates a redundant global-memory read.
3. **Python orchestration elimination.** Adaptive selection, mask construction, and small elementwise kernels (∼40 ms total) should move to fused CUDA kernels or a compiled graph.

None of these affect the certification guarantee. With a tensor-core kernel and fused scoring, the projected overhead is ∼1.3–1.5× dense.

9.8 Numerical Precision of the Attention Kernel

The fused Triton kernel uses explicit FP32 multiply-accumulate on CUDA cores for the $q \cdot k$ dot product, while PyTorch's `scaled_dot_product_attention` dispatches to Flash Attention, which uses tensor-core TF32/FP16 matmul with FP32 accumulation. These two computational paths are mathematically equivalent but not numerically identical: different rounding in the inner product produces ∼1e-7 max absolute difference per block per head.

For language modelling, this difference is negligible—it vanishes into the per-token log-probability noise. For retrieval-adversarial benchmarks (NIAH), where a single token's attention weight determines whether the model "finds" the needle, the compounded rounding difference across 32 layers can flip marginal retrievals.

We mitigate the accumulation component by maintaining the online softmax scalars (m, ℓ) in FP64 precision; the d-dimensional output accumulator o remains FP32. On H100/Blackwell GPUs, Triton supports FP64 arithmetic; the cost is two FP64 scalar updates per block per head (running max m and log-sum-exp normaliser ℓ), which is negligible relative to the memory-bandwidth cost of loading keys and values. Empirically, FP64 accumulators recover ∼2pp on NIAH 8K (Table 14).

The remaining ∼4pp gap is attributable to the matmul implementation (CUDA cores vs. tensor cores), not the quantisation scheme. A tensor-core-native kernel would close this gap. This does not affect the certification guarantee: Rung 3/4 fallbacks bypass the Triton kernel entirely and call `torch.scaled_dot_product_attention` (Section 3.4).

10 Discussion

The primary contribution of this work is not compression itself—KV cache quantisation is well-established—but the *certification framing*: formal per-head, per-step error bounds coupled with runtime monitoring and unconditional FP16 fallback. The tiered cache provides the compression savings of INT8 keys and INT4 values (44% Tier-1 VRAM reduction; runtime savings depend on the FP16 key scratch buffer needed for the GQA union working set—see Table 12) while retaining the error profile of full-precision attention as a guaranteed fallback.

The cost of the guarantee. The certified system runs at $\sim$2.2$\times$ the latency of dense Flash Attention across 16K–64K context (Table 11). This overhead was collapsed from an initial 9.4$\times$ through implementation changes that did not affect the error bounds or fallback logic (Table 10), confirming it is an implementation artefact rather than an inherent cost of the certification architecture. Retaining FP16 originals in system RAM (Tier-2) costs $\sim$16 GB at 128K context. The fallback rungs trigger rarely: Rung 3 on $\sim$0.12% of head-step pairs (NIAH), Rung 2 on $<$2% of blocks.

When to use this system. Table 13 positions the tiered certified cache relative to dense FP16 and standard (uncertified) KV quantisation. The system's value proposition is strongest when the KV cache does not fit in VRAM at full precision and quality regressions are unacceptable—i.e., when the alternative to certified quantisation is either offloading (slow) or uncertified quantisation (risky). It is not the right choice when latency is the binding constraint and dense FP16 fits comfortably.

Table 13: Deployment regime comparison. "Standard KV quant" refers to methods without runtime error bounds or FP16 fallback (e.g., KIVI, KVQuant, per-tensor INT8).

Regime	Dense FP16	Standard quant	This paper
KV fits in VRAM, latency-critical	Best	Viable	Viable (2.2$\times$)
KV does not fit in VRAM	Fails / offloads	Works, no guarantee	Works + fallback
Quality regression intolerable	Expensive	Risky	Best fit
Batch-serving throughput	Strong	Strong	$\sim$0.5$\times$ dense

Two-term independence. The error decomposition's most practically useful property is that key compression error and value compression error are independent—they have separate bounds, separate monitors, and separate escalation paths. The tolerance sweep (varying v_{tol} from 0.5 to 0.05) confirms this empirically: the adaptive top-K selection (K^*) is invariant to value tolerance changes, while η-driven value escalation scales with $1/v_{\text{tol}}$. This decoupling simplifies configuration: the key-side and value-side precision budgets can be tuned independently.

Non-monotonicity of precision vs. retrieval accuracy. The NIAH ablation (Section B) reveals that increasing the FP16 key budget does not monotonically improve retrieval accuracy. The mechanism is attention dilution: promoting tail blocks to FP16 corrects their scores upward, increasing their softmax share at the expense of the needle token's share. This is a general property of non-uniform precision attention, not specific to our system. The ranking-consistency check (Section 6.1) detects this condition at runtime and triggers per-head fallback to dense attention, converting a potential quality regression into a bounded compute cost.

Statistical power and the NIAH result. The initial 30-trial NIAH evaluation appeared to show a fixed $-$6.7pp gap. The powered 100-trial follow-up at 8K reveals this was noise: $\Delta=-2$pp with a CI that comfortably includes zero. This underscores the importance of adequate statistical

power when evaluating retrieval benchmarks, where per-trial variance is high and effect sizes are small. The development-time ablation (Appendix B) correctly identified three sources of variance; the final system's aggregate effect on NIAH accuracy is within noise at the operating point.

Conservative mass estimation. The tight bound on INT8 mass overestimation is $e^{2\Delta} \approx 1.43\times$ (Theorem 4.4); the implementation uses $e^{3\Delta} \approx 1.72\times$ to accommodate the optional INT8×INT8 tensor-core query scoring path (Section 5.2). This means the selector promotes more blocks to FP16 than strictly necessary under FP16 query scoring, wasting page-in bandwidth but not introducing error.

Architecture independence. The tiered cache, error decomposition, and fallback ladder are agnostic to the model architecture. They require only standard scaled dot-product attention with a block-organised KV cache. The per-channel INT8 key quantisation and per-group INT4 value quantisation adapt to different head dimensions and value distributions without model-specific tuning.

11 Limitations

The certified guarantee is *per-head, per-step*: it bounds $\|O_{\text{quant}} - O_{\text{ref}}\|$ (Section 4.1), not the distance to O_{dense}. It does not provide an end-to-end model-quality guarantee. On NIAH, the per-step perturbations are within the certified bound, and the powered 100-trial evaluation shows no statistically significant quality loss ($\Delta = -2$pp, $p = 0.38$). However, the development-time ablation (Table 14) demonstrates that individual precision components (INT4 values, INT8 tail keys, kernel numerical path) each contribute measurable variance at the per-trial level. The aggregate effect cancels to noise at the operating point, but could become significant on tasks with sharper retrieval requirements or longer autoregressive chains. The aggregate effect on model quality must therefore be assessed empirically per deployment.

The system RAM requirement for Tier 2 (FP16 originals) equals the dense KV cache size—e.g., ∼16 GB at 128K context on LLaMA 3.1-8B. This is the price of the unconditional fallback guarantee. Deployments with limited system RAM must either accept the system RAM cost or forgo the fallback (losing the "certified" property and reverting to standard quantisation).

The runtime overhead (∼2.2× dense; Section 9.7) is dominated by the attention kernel and Python orchestration, not the certification logic. At 128K, dense prefill completes in ∼22 s but certified decode is prohibitively slow with the current kernel; full 128K evaluation is ongoing. GQA union saturation (Table 12) means the FP16 key working set covers 87% of the corpus at 64K, requiring a corpus-sized scratch buffer that consumes the Tier-1 VRAM savings; meaningful VRAM savings (>10%) require 128K+ context.

The Triton attention kernel computes dot products on CUDA cores rather than tensor cores, producing slightly different rounding than Flash Attention (Section 9.8). The development-time ablation (Table 14) attributes ∼4pp to this kernel path; however, the powered 100-trial evaluation shows no statistically significant aggregate effect at the operating point. A tensor-core-native kernel would eliminate this source of variance entirely.

The value group-size cliff between $g = 16$ (near-lossless) and $g = 32$ (catastrophic, +26.4 Δppl) means there is no intermediate operating point for INT4 values at $d = 128$. Deployments requiring finer-grained value compression tradeoffs would need INT8 values (not currently implemented in the kernel) or a different head dimension.

Greedy-decode degeneracy at long context. While teacher-forced perplexity is clean at all tested context lengths (4K–32K), free (greedy argmax) continuation at 16K+ exhibits repetition and degeneracy in the generated text. This phenomenon is not observed at 8K and does not

affect the perplexity evaluation (which is teacher-forced), but it indicates that the per-step error bounds, while satisfied, may compound over long autoregressive chains in a way that degrades coherence. The per-head, per-step guarantee does not compose into an end-to-end generation-quality guarantee; this is the fundamental gap between "certified attention" and "certified generation." We note this as an important direction for future work.

12 Conclusion

We presented a tiered KV cache architecture that stores INT8 keys and INT4 values in GPU memory (Tier-1: 56% of FP16; runtime VRAM includes an additional FP16 key scratch buffer sized by the GQA working-set union) while retaining full-precision originals in system RAM (Tier-2) for runtime fallback. A two-term error decomposition provides independent, per-head, per-step bounds on key compression error and value compression error, each with its own runtime monitor and escalation path. An adaptive top-K precision selector promotes high-mass blocks to FP16 keys based on the actual attention pattern at each decode step, and a four-rung fallback ladder ensures that the worst case is the Dense baseline's own `torch.scaled_dot_product_attention` code path with unmodified FP16 KV, not a degraded approximation.

The system introduces a ranking-consistency check that detects when INT8 scoring noise distorts the attention distribution and triggers per-head fallback to dense attention. This mechanism addresses a non-obvious failure mode we identified during evaluation: on retrieval-adversarial benchmarks, non-uniform precision across the sequence can cause attention dilution that degrades accuracy even when per-step error bounds are satisfied.

On language modelling (PG-19), the certified system matches dense perplexity within measurement noise across all tested context lengths (4K–32K). On retrieval tasks (NIAH, 100 paired trials at 8K), the certified system is statistically indistinguishable from dense; at 32K, both systems tie at 70% accuracy. We observe and transparently report a residual kernel-path gap attributable to numerical precision (CUDA-core vs. tensor-core matmul rounding), which is a property of the Triton implementation rather than the quantisation scheme. The ranking-consistency check and FP64 accumulators together recover the majority of this gap; the remainder would be closed by a tensor-core-native kernel.

The certified system runs at $\sim$2.2$\times$ the latency of dense Flash Attention across 16K–64K context, with the overhead confirmed to be an implementation artefact (collapsed from 9.4$\times$ via kernel and cache changes that did not affect the bounds; Section 9.7). GQA union saturation means that meaningful VRAM savings require 128K+ context (Table 12).

The approach requires no model retraining or dataset-specific calibration and applies to any transformer with standard attention.

References

[1] Hooper, C., Kim, S., Mohammadzadeh, H., et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *NeurIPS*, 2024.

[2] Liu, Z., Yuan, J., Jin, H., et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *ICML*, 2024.

[3] Zandieh, A., Rajput, S., Daliri, A., and Hein, M. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *ICML*, 2024.

[4] Zhang, X., et al. Hardware-aware tuning-free quantization of KV cache for large language models. *arXiv preprint*, 2024.

[5] Gao, Z., et al. HACK: Homomorphic acceleration via compression of the key-value cache for disaggregated LLM inference. *SIGCOMM*, 2025.

[6] Wei, Y., et al. VecInfer: Efficient LLM inference with low-bit KV cache via outlier-suppressed vector quantization. *arXiv preprint arXiv:2510.06175*, 2025.

[7] Ye, Z., et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *MLSys*, 2025.

[8] Zhang, Z., Sheng, Y., Zhou, T., et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

[9] Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *ICLR*, 2024.

[10] Singhania, A., et al. BLASST: Block-level adaptive structured sparse attention for LLM inference. *arXiv preprint*, 2024.

[11] Wang, T., et al. PSA: Progressive sparse attention for long-context inference. *arXiv preprint*, 2024.

[12] Li, Y., et al. Twilight: Adaptive attention sparsity with hierarchical top-p pruning. *NeurIPS*, 2025.

[13] Chen, X., et al. SpargeAttn: Accurate sparse attention accelerating any model inference. *arXiv preprint*, 2025.

[14] Chen, R., et al. A mathematical theory of top-k sparse attention via total variation distance. *arXiv preprint arXiv:2512.07647*, 2025.

[15] Prabhu, A., et al. vAttention: Dynamic memory management for serving LLMs without PagedAttention. *OSDI*, 2024.

[16] Kwon, W., et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[17] Zhang, Y., et al. Cocktail: Mixed-precision attention for efficient LLM inference. *arXiv preprint*, 2024.

[18] Li, X., et al. TaDA: Token-aware dynamic attention for efficient transformer inference. *arXiv preprint*, 2024.

[19] Feng, H., et al. Ada-KV: Adaptive KV cache compression with dynamic bit allocation. *arXiv preprint*, 2024.

[20] Dong, H., et al. Don't waste your bits! Variable-length quantization for KV cache via attention entropy. *arXiv preprint*, 2024.

[21] Hsieh, C.-Y., et al. RULER: What's the real context size of your long-context language models? *NeurIPS*, 2024.

[22] Chen, L., et al. TurboQuant: Online vector quantization with near-optimal distortion rate. *ICLR*, 2026. arXiv:2504.19874.

[23] Han, C., et al. JanusQuant: Accurate and efficient 2-bit KV cache quantization for long-context inference. *PPoPP (ACM SIGPLAN)*, 2026.

[24] Guo, T., et al. PackInfer: Compute- and I/O-efficient attention for batched LLM inference. *arXiv preprint arXiv:2602.06072*, 2026.

A Total Variation Bound for Perturbed Softmax

We derive the total variation bound used in the key compression error analysis. The proof proceeds in three steps: bounding individual probability ratios, deriving the full-sequence TV bound, and specialising to the tail-only case used by the adaptive precision selector.

Setup. Let $a_t = \exp(s_t)/Z$ and $a'_t = \exp(s'_t)/Z'$ be two softmax distributions over N tokens, where $|s_t - s'_t| \leq \Delta$ for all t, $Z = \sum_t \exp(s_t)$, and $Z' = \sum_t \exp(s'_t)$.
Lemma A.1 (Probability ratio bound)**.** *For all t: $e^{-2\Delta} \leq a'_t/a_t \leq e^{2\Delta}$.*

Proof. The ratio decomposes as:

$$\frac{a'_t}{a_t} = \frac{\exp(s'_t)}{\exp(s_t)} \cdot \frac{Z}{Z'} = \exp(s'_t - s_t) \cdot \frac{Z}{Z'}$$

Factor 1: per-token logit ratio. Since $|s_t - s'_t| \leq \Delta$, we have $e^{-\Delta} \leq \exp(s'_t - s_t) \leq e^{\Delta}$.

Factor 2: partition function ratio. For the partition functions:

$$Z' = \sum_t \exp(s'_t) \leq \sum_t \exp(s_t + \Delta) = e^{\Delta} Z$$

and symmetrically $Z' \geq e^{-\Delta} Z$. Therefore $e^{-\Delta} \leq Z/Z' \leq e^{\Delta}$.

Multiplying the two factors: $e^{-2\Delta} \leq a'_t/a_t \leq e^{2\Delta}$. □

Lemma A.2 (Full-sequence TV bound)**.** $\mathrm{TV}(a, a') \leq \tanh(\Delta) = \dfrac{e^{2\Delta} - 1}{e^{2\Delta} + 1}$

Proof. The TV is a linear function of the ratios $r_t = a'_t/a_t$. By Lemma A.1, each $r_t \in [e^{-2\Delta}, e^{2\Delta}]$, and the constraint $\sum_t a'_t = 1$ reads $\sum_t a_t r_t = 1$. Since TV is linear in the r_t, its maximum over the feasible polytope is attained at a vertex where each r_t is at one of the extreme values $e^{\pm 2\Delta}$.

Partition tokens into $\mathcal{S}^+ = \{t : r_t = e^{2\Delta}\}$ and $\mathcal{S}^- = \{t : r_t = e^{-2\Delta}\}$. Let $p = \sum_{t \in \mathcal{S}^+} a_t$. The normalisation constraint gives:

$$p \cdot e^{2\Delta} + (1 - p) \cdot e^{-2\Delta} = 1 \implies p = \frac{1 - e^{-2\Delta}}{e^{2\Delta} - e^{-2\Delta}} = \frac{1}{e^{2\Delta} + 1}$$

where the last step uses $(1 - e^{-2\Delta}) = (e^{2\Delta} - 1)e^{-2\Delta}$ and $e^{2\Delta} - e^{-2\Delta} = (e^{2\Delta} - 1)(1 + e^{-2\Delta})$.

Computing the TV at this vertex:

$$\mathrm{TV} = \sum_{t \in \mathcal{S}^+} a_t (e^{2\Delta} - 1) = p\,(e^{2\Delta} - 1) = \frac{e^{2\Delta} - 1}{e^{2\Delta} + 1} = \tanh(\Delta)$$

This bound is tight (attained by the above construction), independent of N, and for small Δ satisfies $\tanh(\Delta) \approx \Delta$. □

Lemma A.3 (Tail-restricted TV bound)**.** *Let $\mathcal{F}$ (FP16 set) and $\mathcal{T}$ (INT8 tail set) partition the N tokens, with scores in $\mathcal{F}$ computed exactly (zero quantisation error) and scores in $\mathcal{T}$ perturbed by at most Δ. Let $\alpha_{\mathcal{T}} = \sum_{t \in \mathcal{T}} a_t$ denote the true tail mass. Then:*

$$\mathrm{TV}(a, a') \leq \alpha_{\mathcal{T}} \cdot (e^{2\Delta} - 1)$$

Proof. Write $a'_t = \exp(s'_t)/Z'$. For FP16 tokens $t \in \mathcal{F}$, $s'_t = s_t$ (no quantisation error), so:

$$a'_t = \frac{\exp(s_t)}{Z'} = a_t \cdot \frac{Z}{Z'}$$

For tail tokens $t \in \mathcal{T}$, the logits are perturbed: $|s_t - s'_t| \leq \Delta$.

The partition function ratio satisfies (using the decomposition $Z' = Z'_{\mathcal{F}} + Z'_{\mathcal{T}}$):

$$\begin{aligned} Z' &= \sum_{t \in \mathcal{F}} \exp(s_t) + \sum_{t \in \mathcal{T}} \exp(s'_t) \\ &\leq Z_{\mathcal{F}} + e^{\Delta} Z_{\mathcal{T}} \\ &= Z + (e^{\Delta} - 1) Z_{\mathcal{T}} \\ &= Z \left(1 + (e^{\Delta} - 1)\alpha_{\mathcal{T}} \cdot \frac{Z}{Z_{\mathcal{F}} + Z_{\mathcal{T}}}\right) = Z(1 + (e^{\Delta} - 1)\alpha_{\mathcal{T}}) \end{aligned}$$

and symmetrically $Z' \geq Z(1 - (1 - e^{-\Delta})\alpha_{\mathcal{T}})$.

For the total variation, we bound $|a_t - a'_t|$ separately on each set.

FP16 tokens ($t \in \mathcal{F}$): $a'_t = a_t \cdot Z/Z'$, so $\sum_{t \in \mathcal{F}} |a_t - a'_t| = (1 - \alpha_{\mathcal{T}}) \cdot |1 - Z/Z'|$. Since $|Z' - Z| = |Z'_{\mathcal{T}} - Z_{\mathcal{T}}| \leq (e^{\Delta} - 1)Z_{\mathcal{T}}$ and $Z' \geq Z_{\mathcal{F}}$ (as $Z'_{\mathcal{T}} \geq 0$):

$$|1 - Z/Z'| = |Z' - Z|/Z' \leq (e^{\Delta} - 1)\, Z_{\mathcal{T}}/Z_{\mathcal{F}} = (e^{\Delta} - 1)\, \frac{\alpha_{\mathcal{T}}}{1 - \alpha_{\mathcal{T}}}$$

Multiplying by $(1 - \alpha_{\mathcal{T}})$:

$$\sum_{t \in \mathcal{F}} |a_t - a'_t| \leq \alpha_{\mathcal{T}}(e^{\Delta} - 1) \tag{17}$$

Tail tokens ($t \in \mathcal{T}$): For each tail token, $a'_t/a_t = \exp(s'_t - s_t) \cdot Z/Z'$. Since $|\delta_t| \leq \Delta$, the first factor satisfies $\exp(\delta_t) \leq e^{\Delta}$. For the second factor, $Z' \geq Z_{\mathcal{F}} + e^{-\Delta} Z_{\mathcal{T}} = Z - (1 - e^{-\Delta})Z_{\mathcal{T}} = Z(1 - (1 - e^{-\Delta})\alpha_{\mathcal{T}})$. Since $\alpha_{\mathcal{T}} \leq 1$, we have $(1 - e^{-\Delta})\alpha_{\mathcal{T}} \leq 1 - e^{-\Delta}$, so $Z' \geq Ze^{-\Delta}$, giving $Z/Z' \leq e^{\Delta}$. Therefore $a'_t/a_t \leq e^{2\Delta}$, and symmetrically $a'_t/a_t \geq e^{-2\Delta}$. Since $|1 - r| \leq \max(e^{2\Delta} - 1, 1 - e^{-2\Delta}) = e^{2\Delta} - 1$ for $r \in [e^{-2\Delta}, e^{2\Delta}]$:

$$\sum_{t \in \mathcal{T}} |a_t - a'_t| = \sum_{t \in \mathcal{T}} a_t\, |1 - a'_t/a_t| \leq \alpha_{\mathcal{T}}(e^{2\Delta} - 1) \tag{18}$$

Combining. Adding (17) and (18):

$$\sum_t |a_t - a'_t| \leq \alpha_{\mathcal{T}}(e^{\Delta} - 1) + \alpha_{\mathcal{T}}(e^{2\Delta} - 1) = \alpha_{\mathcal{T}}(e^{2\Delta} + e^{\Delta} - 2)$$

Since $e^{\Delta} \leq e^{2\Delta}$ for all $\Delta \geq 0$, we have $e^{2\Delta} + e^{\Delta} - 2 \leq 2(e^{2\Delta} - 1)$, and therefore:

$$\mathrm{TV}(a, a') = \tfrac{1}{2} \sum_t |a_t - a'_t| \leq \alpha_{\mathcal{T}} \cdot (e^{2\Delta} - 1)$$

All inequalities are non-asymptotic, holding for every $\Delta \geq 0$ and $\alpha_{\mathcal{T}} \in [0, 1]$. □

Combining with Theorem 4.4. Let $\hat{\alpha}_{\mathcal{T}} = \sum_{b \notin \mathcal{F}} p_b^{\text{int8}}$ denote the INT8-estimated tail mass after all clamping and Rung 1 expansion. The adaptive precision selector targets $\hat{\alpha}_{\mathcal{T}} \leq 1 - \tau_{\text{cov}}$, but $K_{\max}$ clamping may prevent this; the bound uses the achieved $\hat{\alpha}_{\mathcal{T}}$. By Theorem 4.4, the tight bound on true tail mass is $\alpha_{\mathcal{T}} \leq e^{2\Delta}\, \hat{\alpha}_{\mathcal{T}}$. Substituting into Lemma A.3:

$$E_{\text{key}} \leq 2V_{\max} \cdot e^{2\Delta}\, \hat{\alpha}_{\mathcal{T}} \cdot (e^{2\Delta} - 1)$$

which is the bound stated in Equation (12). When the coverage target is met ($\hat{\alpha}_{\mathcal{T}} = 1 - \tau_{\text{cov}}$), this reduces to $2V_{\max} \cdot e^{2\Delta}(1 - \tau_{\text{cov}})(e^{2\Delta} - 1)$. The implementation substitutes $e^{3\Delta}$ to accommodate optional INT8 query scoring (Section 5.2).

B NIAH Precision Ablation

During development, we observed a counter-intuitive non-monotonicity on NIAH 8K: increasing the FP16 key budget did not monotonically improve retrieval accuracy. We conducted a systematic ablation to decompose the sources of the tiered–dense gap. Table 14 reports results from the pre-FP64 kernel; the FP64-patched kernel recovers an additional ∼2pp.

Table 14: NIAH 8K precision ablation (pre-FP64 development kernel, 30 trials). Each row isolates one precision component. The dense baseline here (72%) differs from the final results in Table 1 because this ablation was run during development on an earlier kernel revision with a different random seed schedule; the relative gaps between configurations are the diagnostic signal, not the absolute values.

Configuration	Dense	Tiered	Gap	What changed
$k_{\max}$=128, INT4 values	72%	62%	−10	Operating point
$k_{\max}$=∞, INT4 values	72%	56%	−16	Uncapped top-K
$k_{\max}$=∞, FP16 values	72%	62%	−10	Values isolated
All FP16 (keys+values, FP32 acc)	72%	66%	−6	Keys isolated
All FP16, FP64 accumulators	72%	68%	−4	Accumulators isolated
Kernel bypassed (torch SDPA)	72%	72%	0	Kernel path isolated

The ablation reveals three independent sources of the tiered–dense gap on NIAH:

INT4 value quantisation (∼6pp at $k_{\max}$=∞). INT4 values with g=16 introduce ∼5% relative reconstruction error. For perplexity this is negligible ($\Delta\text{ppl} < 0.005$), but NIAH's reliance on exact value content for the needle tokens makes it more sensitive. The value escalation mechanism (Rung 2, triggered by η_b) addresses this for blocks with high reconstruction error.

INT8 key scoring on tail blocks (∼4pp). With the operating-point cap $k_{\max}$=128, roughly 15% of blocks remain on INT8 keys. Their slightly perturbed scores shift the softmax denominator, redistributing attention mass away from the needle. Paradoxically, promoting *more* blocks to FP16 ($k_{\max}$=∞, K^*≈221) makes this *worse*: the newly promoted blocks' FP16 scores are more accurate (higher) than their INT8 estimates, increasing their softmax share at the needle's expense. The ranking-consistency check (Section 6.1) detects and corrects this per-head.

Kernel numerical path (∼4pp residual). Our Triton kernel computes $q \cdot k$ via explicit FP32 multiply-accumulate on CUDA cores, while `torch.scaled_dot_product_attention` dispatches to Flash Attention using tensor-core TF32/FP16 matmul with FP32 accumulation. The different rounding paths produce ∼1e-7 max absolute difference per step, which compounds through 32 layers of autoregressive decoding. FP64 online-softmax accumulators recover ∼2pp of this gap (Section 9.8). The remaining ∼4pp is an inherent property of the matmul implementation, not the quantisation scheme, and would be present in any custom Triton attention kernel that does not use tensor cores.

Implication for the certified claim. The certification guarantee (Section 4) bounds the per-head, per-step output perturbation from quantisation. The NIAH ablation shows that on retrieval-adversarial benchmarks, even perturbations within the certified bound can affect downstream accuracy through the argmax over output logits. This is a known limitation of per-step bounds (Section 11). For language modelling quality (PG-19), where the metric averages over all tokens, the certified system matches dense within measurement noise.